

L-1

FLAT

→ Alphabet - $\{a, b\}$, $\{a, b, c\}$, $\{0, 1\}$, $\{0, 1, \dots, q\} = \Sigma$

→ String - Ex: $a, b, aa, ab, bc, \dots$

→ If the Alphabet set Σ contains 'n' alphabets i.e. the no. of alphabets in the set are 'n' (denoted by $|\Sigma|$) then the no. of possible strings of length 'n' possible over Σ is $|\Sigma|^n$.

→ Language = collection of strings

Let, $\Sigma = \{a, b\}$ then Language

$L_1 = \{\text{set of all strings of length 2}\} \rightarrow \text{Finite Language}$
 $= \{aa, ab, ba, bb\}$

$L_2 = \{\text{set of all strings of length 3}\} \rightarrow \text{Finite Language}$
 $= \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

$L_3 = \{\text{set of all strings where each string starts with 'a'}\}$
 $= \{a, aa, aaa, ab, abb, aba, \dots\} \rightarrow \text{Infinite Language}$

Powers of Σ

Let the Alphabet set be $\Sigma = \{a, b\}$

$\Sigma^1 = \{\text{set of all strings of Length exactly one}\} = \{a, b\}$

$\Sigma^2 = \Sigma \cdot \Sigma = \{\text{set of all strings of Length exactly 2}\} = \{aa, ab, ba, bb\}$

$\Sigma^3 = \Sigma \cdot \Sigma \cdot \Sigma = \{\text{set of all strings of length exactly 3}\} = \{aaa, aab, aba, abb, baa, bab, bba, bbb\} = 8$

$\Sigma^0 = \{\text{set of all strings of length '0'}\} = \{\epsilon\}$ $\swarrow$ Epsilon = Null string $[|\epsilon| = 0]$

$\Sigma^* = \{\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \dots\} = \{\{\epsilon\} \cup \{a, b\} \cup \{aa, ab, ba, bb\} \dots\}$

$= \{\text{set of possible strings over } (a, b) \text{ of all lengths}\}$

$= \{\text{Universal set}\}$

PRACTICAL SCENARIO

(2)

Now, consider the C language $\Sigma = \{a, b, \dots, z, A, B, \dots, Z, 0, \dots, 9, +, \dots, *, \dots\}$

Now, `void main()`

```

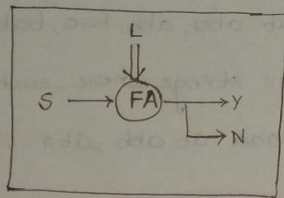
{
    int a, b;
    !
}
    
```

program in 'C' but in TAC it is a string

Now, C-programming language = {set of all "valid" programs}

= $\{P_1, P_2, P_3, \dots\}$ (P_n)

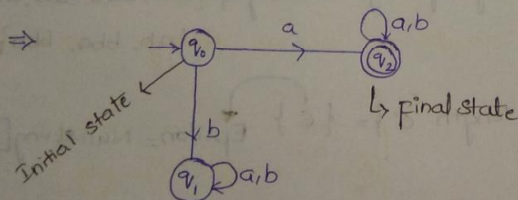
Now, Given any string and to find out whether the string belongs to the language is a heavy task if the language is infinite.



CONSTRUCTION OF FINITE AUTOMATA

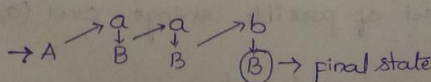
① $L_1 = \{\text{set of all strings which start with 'a'}\}$

= $\{a, aa, ab, aaa, \dots\}$

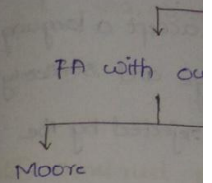


States = $\{q_0, q_1, q_2\}$

Now, Given string, = a a b

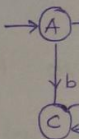


Classification



DFA

DFA = $(Q, \Sigma, \delta, q_0, F)$



② Construct

Length 2?

$\Sigma = \{a, b\}$

$L = \{aa, ab, ba, bb\}$

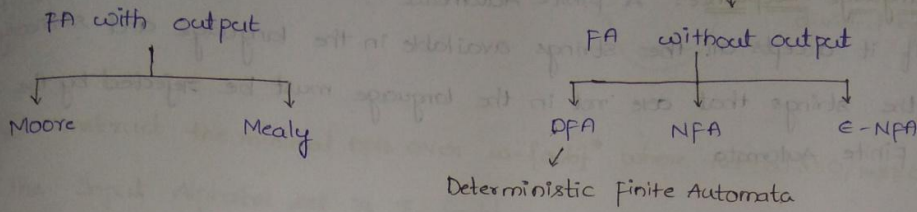
Given string

②

Classification of the Finite Automata.

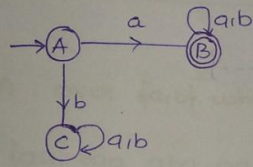
③

Finite Automata



DFA

$$DFA = (Q, \Sigma, \delta, q_0, f)$$



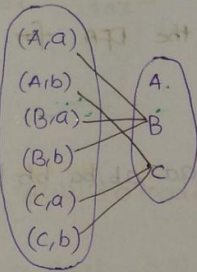
$$Q = \{\text{set of all states}\} = \{A, B, C\}$$

$$\Sigma = \{\text{set of Alphabets}\} = \{a, b\}$$

$$q_0 = \{A\}, f = \{B\}$$

$$\text{NOW, } \delta = Q \times \Sigma \rightarrow Q$$

$$\{A, B, C\} \times \{a, b\}$$

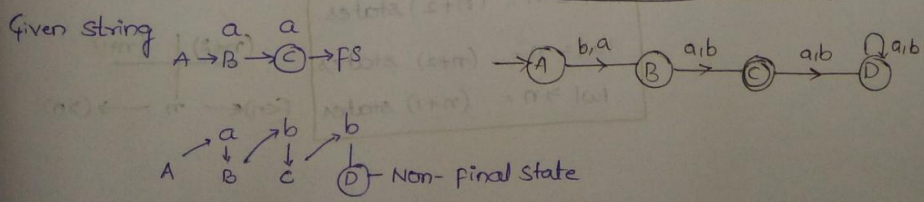
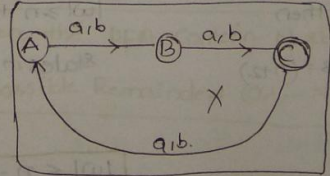


② construct a DFA that accepts set of all strings over $\{a,b\}$ of

Length 2? $|w| = 2$.

$$\Sigma = \{a, b\}$$

$$L = \{aa, ab, ba, bb\}$$



⇒ String Acceptance: Scan the entire string, if we reach a final state from the initial state then the string is accepted. (4)

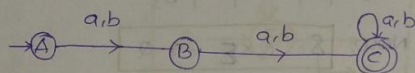
⇒ Language Acceptance: A finite Automata is said to accept a language, if it accepts all the strings available in the language and similarly the strings that are not in the language must be rejected by the finite Automata.

L-2 CONSTRUCTION OF MINIMAL DFA

construct a DFA for set of all strings over $\{a, b\}$ such that length of the string is atleast 2? $|w| \geq 2$

Sol Now, $\Sigma = \{a, b\}$

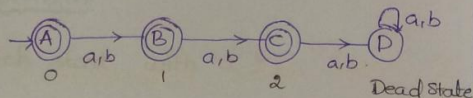
$L = \{aa, bb, aab, aaa, bbb, baa, \dots\}$



Now, construct the DFA for which $|w| \leq 2$

$\Sigma = \{a, b\}$

$L = \{\epsilon, a, b, aa, ab, ba, bb\}$



Now, from the above problems the conclusions that can be drawn are

$$|w| = 2$$

$$|w| \geq 2$$

$$|w| \leq 2$$

If $|w| = n$ then

No. of states = $(n+2)$

$|w| \geq n$ then no. of

states in DFA is $(n+1)$

$|w| \leq n$ the no. of

states = $(n+2)$

$$|w| \leq n = (n+2) \text{ states}$$

$$|w| = n = (n+2) \text{ states}$$

$$|w| \geq n = (n+1) \text{ states}$$

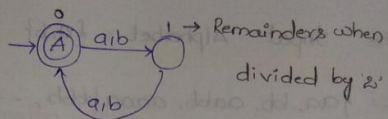
$$\begin{array}{c} (n+2) \quad | \quad (n+1) \\ \hline \leftarrow n \rightarrow \end{array}$$

4

5) construct the minimal DFA over $\{a,b\}$ where $|w| \bmod 2 = 0$.

The Input Alphabet set is $\Sigma = \{a,b\}$

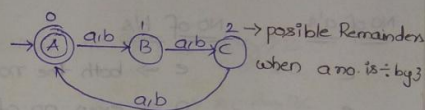
$L = \{aa, ab, ba, bb\}$ and ϵ



6) construct the minimal DFA over $w = \{a,b\}^*$ where $|w| \bmod 3 = 0$ or $|w| \equiv 0 \bmod 3$

The Input Alphabet set is $\Sigma = \{a,b\}$

$L = \{aaa, bbb, abb, baq, aba, bab, \dots\}$

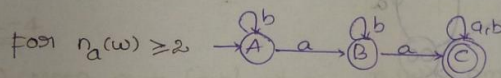
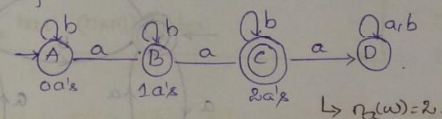


If $|w| \bmod n = 0$ then the DFA has 'n' states.

7) CMDFA over $\{a,b\}$ where no. of a's in $w = 2 \Rightarrow \{n_a(w) = 2\}$

$L = \{aa, baq, aba, aab, aabbb, \dots\}$

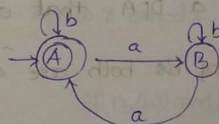
The Input Alphabet set $\Sigma = \{a,b\}$



8) CMDFA where $n_a(w) \bmod 2 = 0$ or $n_a(w) \equiv 0 \bmod 2$

$L = \{aa, aab, baq, aab, aba, \dots\}$

The Input Alphabet set is $\Sigma = \{a,b\}$



If $|w| \bmod n \equiv K \bmod N$ then the DFA contain N states and each state represent the possible Remainders $(0, 1, \dots, N-1)$.

⑨ CMDFA over $\{a, b\}$ where $n_a(w) \equiv 0 \pmod 2$ and $n_b(w) \equiv 0 \pmod 2$. ⑩

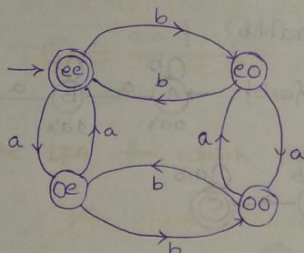
The Input Alphabet = $\{a, b\}$

$L = \{aa, bb, aabb, aaaa, bbbb, \dots \text{ and } \epsilon\}$

The entire Group of strings for the above language can be classified into 4 categories. They are

<u>No. of a's</u>	<u>No. of b's</u>
e	$e \Rightarrow$ both the no. of a's and no. of b's are even
e	$o \Rightarrow$ even no. of a's and odd no. of b's
o	$e \Rightarrow$ odd " " " " even " " "
o	$o \Rightarrow$ " " " " odd " " "

$\therefore$ Therefore the DFA contains 4 states representing each category.



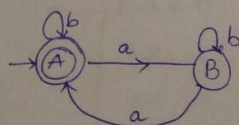
L-3 CROSS PRODUCT METHOD

⑩ construct a DFA that accepts strings over $\{a, b\}$ where no. of a's and no. of b's both are even?

The Input Alphabet $\Sigma = \{a, b\}$ $w = (a, b)^*$ $n_a(w) \equiv 0 \pmod 2$

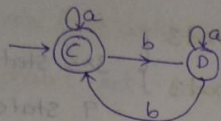
$n_b(w) \equiv 0 \pmod 2$

Now, construct the DFA for counting a's



⑥

Similarly the DFA for counting the b's will be

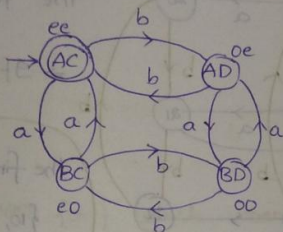


⑦

classified

Now, the cross product of the two Automaton will be

$$\{A, B\} \times \{C, D\} = \{AC, AD, BC, BD\}$$



Now, $AC \xrightarrow{a} ? \Rightarrow A \xrightarrow{a} B$
 $\Rightarrow AC \xrightarrow{a} BC$

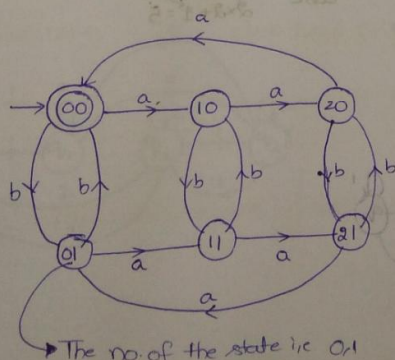
$AC \xrightarrow{b} ? \Rightarrow A \xrightarrow{b} C$
 $\Rightarrow AC \xrightarrow{b} AD$

If the 1st DFA has 'n' states and 2nd DFA has 'm' states then the cross product of DFA₁ and DFA₂ has (n×m) states.

L-4

⑪ CMDFA over $\{a, b\}$ where $n_a(w) \geq 0 \pmod 3$ and $n_b(w) \geq 0 \pmod 2$.

The Input Alphabet $\Sigma = \{a, b\}$ $w = (a, b)^*$ $n_a(w) \geq 0 \pmod 3$ and $n_b(w) \geq 0 \pmod 2$



If $n_a(w) \pmod 3 \geq n_b(w) \pmod 2$ then final states are

$$\{10, 20, 21, 11\}$$

$$\text{If } n_a(w) \pmod 3 = n_b(w) \pmod 2$$

$$\{00, 11\} = \text{final states.}$$

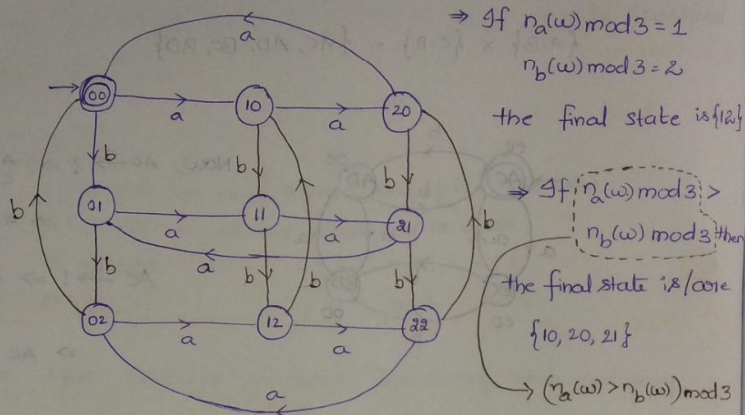
→ The no. of the state is 0, 1

represent that to reach that state we should cross 0 a's and 1 b's.

L-5

- ⑫ CMDFA over $\{a, b\}$ where $n_a(w) \equiv 0 \pmod 3$
 $n_b(w) \equiv 0 \pmod 3$ } 3×3 States
 $= 9$ States

The Input Alphabet set = $\{a, b\}$



L-6

- ⑬ CMDFA over $\{0, 1\}$ which when Interpreted as Binary number is ≥ 10

The Input Alphabet set = $\{0, 1\}$

1- Unary

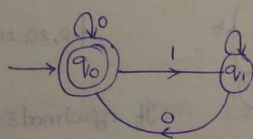
2- Binary - 0, 1

3- Ternary - 0, 1, 2

!

10- Decimal - 0, 1 -- 9

16- Hexadecimal - 0, 1 -- 15



Now, $(10)_2 = (1 \times 2) + 0 = 2$

Base $(101)_2 = 1 \times 2 + 0 = 2$
 $2 \times 2 + 1 = 5$

L-7

- ⑭ CMDFA over $\{a, b\}$
The Input Alphabet

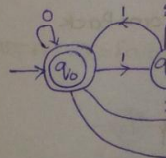


L-8

- ⑮ CMDFA over $\{a, b\}$
The Input Alphabet



Now if $\Sigma = \{0, 1\}$

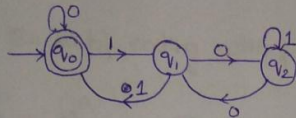


8

L-7

14) CMDFA over $\{0,1\}$ which interpreted as Binary number is $\div 3$ by 3

The Input Alphabet set = $\{0,1\}$



The Transition table is

	0	1
q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

Shortcut method

$$3 = 1$$

$$3 = 2$$

state is $\{1,2\}$

$w \bmod 3 >$

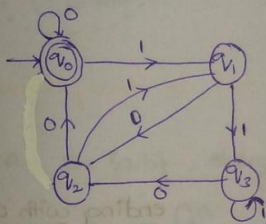
$(w \bmod 3) \bmod 3$ then

state is $\{0,1,2\}$

L-8

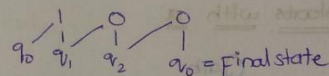
15) CMDFA over $\{0,1\}$ which when interpreted as Binary number is $\div 4$ by 4

The Input Alphabet set = $\{0,1\}$, The Transition table is

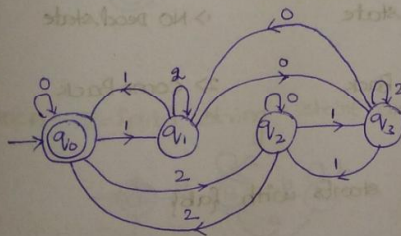


	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_0	q_1
q_3	q_2	q_3

Shortcut



Now if $\Sigma = \{0,1,2\}$ (Ternary numbers) then, the Transition table is.



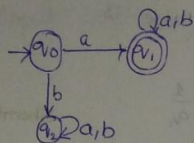
	0	1	2
q_0	q_0	q_1	q_2
q_1	q_2	q_0	q_1
q_2	q_1	q_2	q_0
q_3	q_1	q_2	q_3

Short cut

L-9

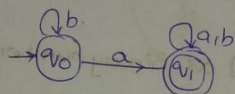
16) CMDFA over $\Sigma = \{a, b\}$ and all the strings start with 'a'?

$L = \{a, aa, ab, aaa, \dots\}$



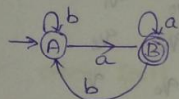
L-10

17) CMDFA, $\Sigma = \{a, b\}$ where each string contains 'a'?



L-11

18) CMDFA $\Sigma = \{a, b\}$ each string ending with 'a'?



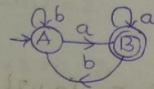
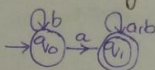
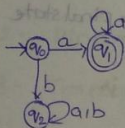
L-12

Now, comparing the above 3 DFA's we get

starts with 'a'

containing 'a'

ending with 'a'



1> Dead state is present

1> NO Dead state

1> NO Dead state

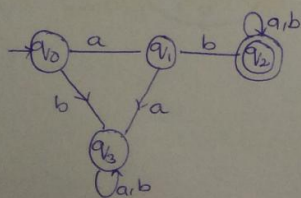
2> No come Back

2> No come Back

2> Come Back

L-13

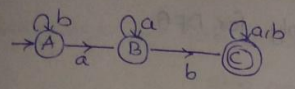
19) CMDFA, $\Sigma = \{a, b\}$ and each string starts with {a,b}



10

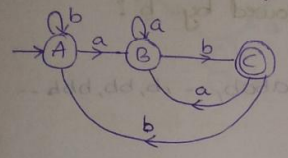
L-14

Q10) CMDFA $\Sigma = \{a, b\}$, Each string containing $\{ab\}$



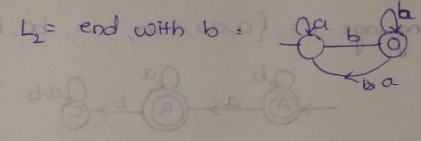
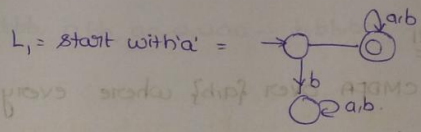
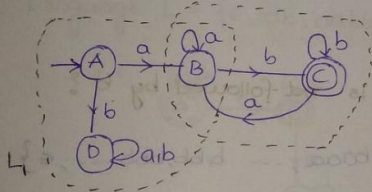
L-15

Q11) CMDFA $\Sigma = \{a, b\}$ ending with $\{ab\}$



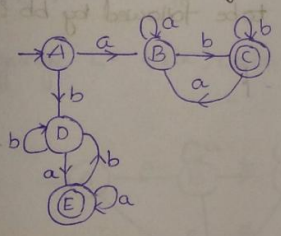
L-16

Q12) CMDFA $\Sigma = \{a, b\}$ start with 'a' and end with 'b'?



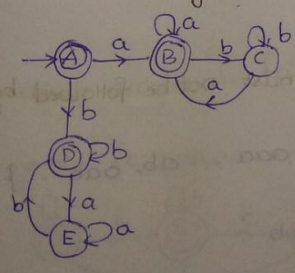
L-17

Q13) CMDFA $\Sigma = \{a, b\}$, string start and end with different symbol?



L-18

Q14) CMDFA $\Sigma = \{a, b\}$ string start and end with same symbol?



Two Languages are said to be Complement to each other when $L_2 = \Sigma^* - L_1$

where L_1, L_2 are defined on same Σ

19 COMPLEMENTATION OF DFA

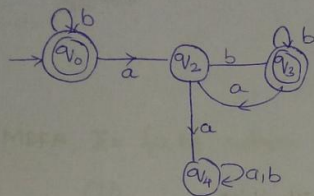
1) The complementation method is applicable only for DFA.

$$2) (Q, \Sigma, \delta_0, q_0, f) \xrightarrow{\text{comp}} (Q, \Sigma, \delta, q_0, Q - F)$$

L-20

25) CMDFA over $\{a, b\}$ where every 'a' is followed by 'b'?

sol

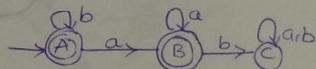


$$L = \{\epsilon, ab, abab, \dots, b, bb, bbb, \dots\}$$

L-21

26) CMDFA over $\{a, b\}$ where every 'a' is not followed by 'b'?

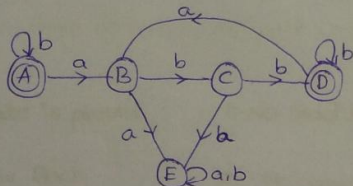
$$\text{Language } L = \{a, aa, aaa, \dots, ba, baa, baaa, \dots, bbb, bbbb, \dots, \epsilon\}$$



L-22

27) CMDFA over $\{a, b\}$ where every 'a' has to be followed by 'bb'?

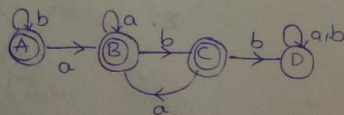
$$\text{Language } L = \{\epsilon, b, bbb, \dots, abb, babb, \dots\}$$



L-23

28) CMDFA over $\{a, b\}$ where every 'a' must not be followed by 'bb'?

$$\text{Language } L = \{\epsilon, b, bb, bbb, \dots, a, aa, aaa, \dots, ab, aab, \dots\}$$



L-24

29) CMDFA which

The Language $L =$

L-25

30) CMDFA which a

The Language $L =$

L-26

31) CMDFA which

The Language $L =$

L-27

32) CMDFA which

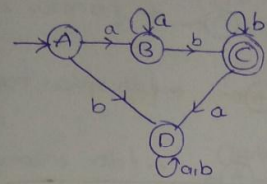
The Language $L =$

12

L-24

27) CMDFA which accepts the Language $L = \{a^n b^m / n, m \geq 1\}$ (13)

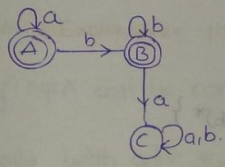
The Language $L = \{ab, aabb, aaabbb, aaaabbbb, \dots\}$
 $abb, aabb, aaabb\}$



L-25

28) CMDFA which accepts $L = \{a^n b^m / n, m \geq 0\}$

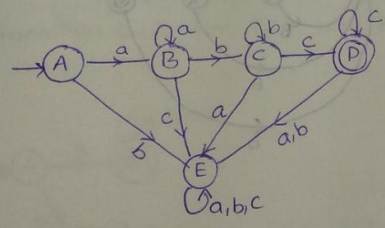
The Language $L = \{\epsilon, ab, aabb, aaabbb, abb, aa, a, aaa, \dots, bbb, bbb, \dots\}$



L-26

31) CMDFA which accepts $L = \{a^n b^m c^l / n, m, l \geq 1\}$

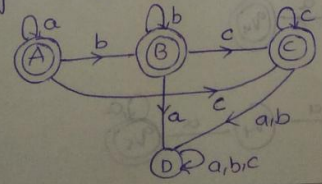
The Language $L = \{abc, aabc, abbc, abcc, \dots\}$



L-27

32) CMDFA which accepts $L = \{a^n b^m c^l / n, m, l \geq 0\}$

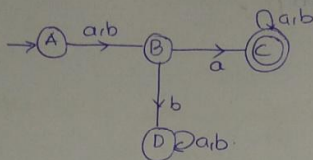
The Language $L = \{\epsilon, a, aa, aaa, \dots, bbb, bbb, \dots, c, cc, cc, \dots, abc, aabcc, \dots, aabb, aaacc, \dots\}$



L-29

(23) CMDFA over $\{a, b\}$ such that the 2nd symbol from the LHS is 'a'?

The language $L = \{aaaa, aa, ba, aab, ba---\}$



If the n th symbol from the LHS is 'a' then the DFA contains $(n+2)$ states

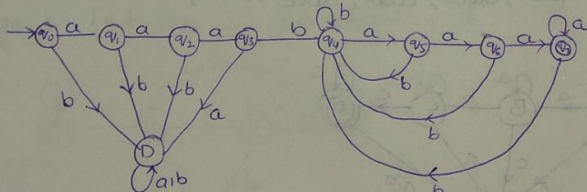
L-30

(24) CMDFA over $\{a, b\}$ where strings are of the form $a^3 b w a^3$ where w is any string over $\{a, b\}$

The language $L = \{a^3 b w a^3 / w \in (a, b)^*\}$

$L = \{a^3 b \in a^3, a^3 b a^3, a^3 b b a^3, a^3 b a a a^3, \dots\}$

The min. length string = $aaabaaaa$

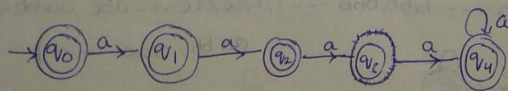


L-32

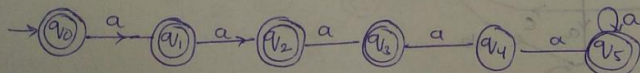
(25) CMDFA over $\{a\}$ such that 1) $\{a^n / n \geq 0, n \neq 3\}$

2) $\{a^n / n \geq 0, n \neq 4\}$

①

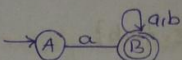


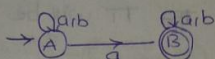
②

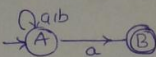


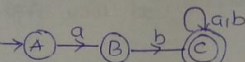
(14)

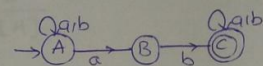
L-34 NFA

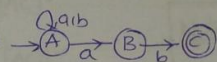
$L_1 = \{ \text{starts with 'a'} \} \Rightarrow$ 

$L_2 = \{ \text{containing a} \} \Rightarrow$ 

$L_3 = \{ \text{ends with a} \} \Rightarrow$ 

$L_4 = \{ \text{start with ab} \} \Rightarrow$ 

$L_5 = \{ \text{contains ab} \} \Rightarrow$ 

$L_6 = \{ \text{End with ab} \} \Rightarrow$ 

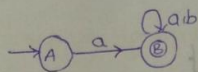
L-35 NFA- DFA CONVERSION

where

The powerfulness (Expressive power) of both NFA and DFA are same because every NFA can be converted to DFA. ($NFA \cong DFA$)

$L_1 = \{ \text{starts with a} \} \Sigma = \{ a, b \}$

NFA for L_1 is.

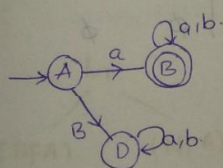


The state Transition table for the above NFA is

	a	b
A	B	$\emptyset$
B	B	B

The state Transition table for DFA will be

	a	b
$\rightarrow A$	B	D
*B	B	B
D	D	D

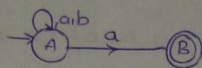


The transition function for NFA is

$$Q \times \Sigma \cup \{ \epsilon \} \rightarrow 2^Q$$

Language $L = \{ \text{ending with } a \}$ $\Sigma = \{a, b\}$

NFA =

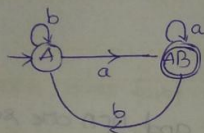


The TT will be

	a	b
A	{AB}	{A}
B	{\phi}	{\phi}

DFA Transition table will be

	a	b
→ A	[AB]	[A]
[AB]	[AB]	[A]
[B]	D	D

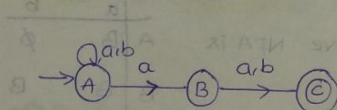


$$\begin{aligned}
 [AB] \rightarrow a &= (A \rightarrow a) \cup (B \rightarrow a) \\
 &= \{A|B\} \cup \{\phi\} \\
 &= \{A, B\} \\
 [AB] \rightarrow b &= (A \rightarrow b) \cup (B \rightarrow b) \\
 &= [A] \cup [\phi] = [A]
 \end{aligned}$$

L-37

$L = \{ \text{all strings where the 2nd symbol from RHS is } a \}$ $\Sigma = \{a, b\}$

NFA = $L = \{aa, ab, aaa, aab, \dots\}$

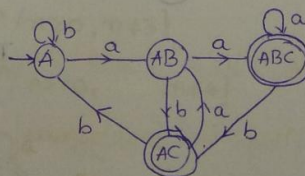


Transition table

	a	b
→ A	{AB}	{A}
B	{c}	{c}
*C	{\phi}	{\phi}

The state Transition table for DFA will be

	a	b
→ [A]	[AB]	[A]
[AB]	[ABC]	[AC]
* [ABC]	[ABC]	[AC]
* [AC]	[AB]	[A]
[B]	[c]	[c]



L-38

$L = \{ \text{All strings} \}$

NFA :



The transition

	a	b
→ [A]	[A]	[A]
[AB]	[A]	[A]
[ABC]	[A]	[A]
[AC]	[A]	[A]
* [ABCD]	[A]	[A]
* [ACD]	[A]	[A]
* [ABD]	[A]	[A]
* [AD]	[A]	[A]

L-39

NFA for string

- a) exactly 2
- b) Atleast 2
- c) Almost 2

(DFA)

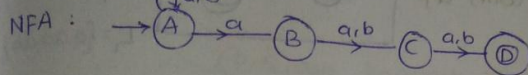
n((DFA))

No. of states
stat

16

L-38

L = { All strings from which 3rd symbol from RHS is a }



The transition table for DFA will be

	a	b
→ [A]	[AB]	[A]
[AB]	[ABC]✓	[AC]✓
[ABC]	[ABCD]✓	[ACD]✓
[AC]	[ABD]✓	[AD]✓
*[ABCD]	[ABCD]	[ACD]
*[ACD]	[ABD]	[AD]
*[ABD]	[ABC]	[AC]
*[AD]	[AB]	[A]

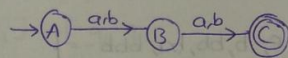
	a	b
→ A	{AB}	{A}
B	{C}	{C}
C	{D}	{D}
*D	{∅}	{∅}

Note : If the minimal NFA contains 'n' states then the ~~minimal~~ DFA contains "2" states (maximum. minimum)

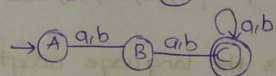
L-39

NFA for strings of length a. exactly 2 b) Atmost 2 c) Atleast 2

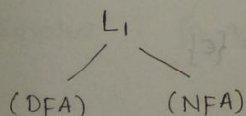
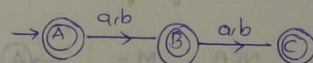
a) exactly 2 L = {aa, ab, ba, bb}



b) Atleast 2 L = {aaa, bbb, aab, abb, bba, bbb, ...}



c) Atmost 2 L = {ε, a, b, ab, ba, bb, aa}



$$n((DFA)) \geq n((NFA))$$

[No of states in DFA] ≥ [No of states in NFA]

Note: The minimum no of states present in the finite Automata(NFA) which accepts set of all strings of length 'n' is 'n' (Only for NFA)

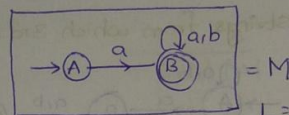
L-40

COMPLEMENTATION OF NFA

(8)

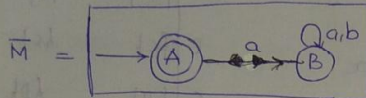
NFA

over $\{a, b\}$ $L = \{ \text{starts with } a \}$

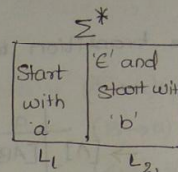


$L_1 = \{a, aa, ab, \dots\}$

Now,



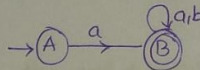
$L_2 = \{\epsilon\}$



Note:

- 1> When we complement the DFA the language will also get complemented
- 2> when we complement the NFA the language may/maynot get complemented.

consider, the NFA



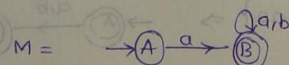
what is the complement of language accepted by this NFA?

$L_1 = \{a, aa, ab, aaa, \dots\}$

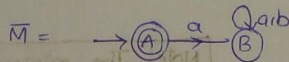
$L_2 = \{\epsilon, b, bb, ba, bbb, \dots\}$

what is the language accepted by complement of the NFA given above?

NFA



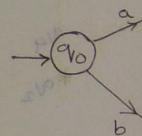
$L_1 = \{a, aa, ab, aaa, \dots\}$



$L_2 = \{\epsilon\}$

L-41 MINIMISATION

Two states 'p'



'0' Equivalence

1 Equivalence

2 Equivalence

3 Equivalence

1> check with

2> if they

MINIMISATION OF DFA

Two states 'p' and 'q' are Equivalent when

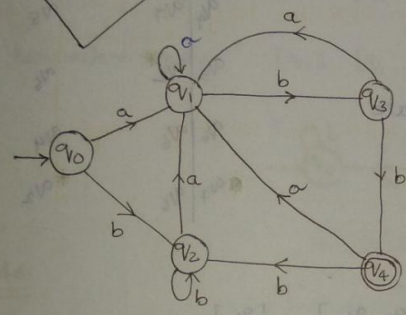
(p, q) are Equivalent if

Input Alphabet

$$\begin{aligned} \delta(p, w) \in F &\Rightarrow \delta(q, w) \in F \\ \delta(p, w) \notin F &\Rightarrow \delta(q, w) \notin F \end{aligned}$$

PARTITIONING METHOD

If $|w|=0 \Rightarrow '0'$ Equivalent
 $|w|=1 \Rightarrow '1'$ Equivalent
 $|w|=n = 'n'$ Equivalent



The Transition table is

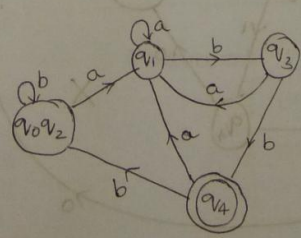
	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	$*q_4$
$*q_4$	q_1	q_2

0 Equivalent = $[q_0 \ q_1 \ q_2 \ q_3] \ [q_4]$

1 Equivalent = $[q_0 \ q_1 \ q_2] \ [q_3] \ [q_4]$

2 Equivalent = $[q_0 \ q_2] \ [q_1] \ [q_3] \ [q_4]$ } Same, so stop here

3 Equivalent = $[q_0 \ q_2] \ [q_1] \ [q_3] \ [q_4]$

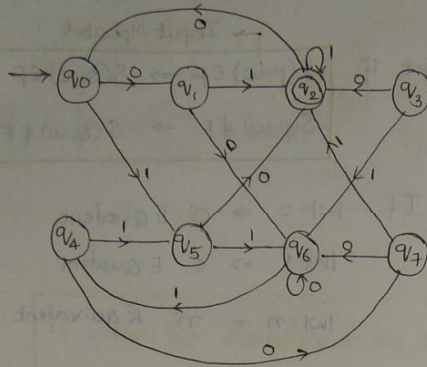


Note: The states that are not reachable from the initial state must be Removed.

- 1> check whether they are in same group in previous Equivalence
- 2> If they are not equal, now check if they are going to same states or not

L-42

Minimise the following DFA



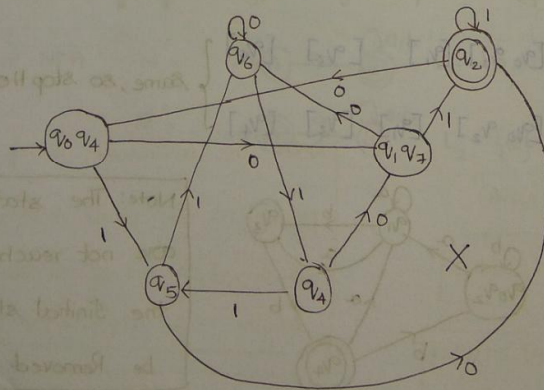
⇒ q_3 is not Reachable from Initial state

0 Equivalent states $[q_0, q_1, q_4, q_5, q_6, q_7]$ $[q_2]$

1 Equivalent states $[q_0, q_4, q_6]$ $[q_1, q_7]$ $[q_5]$ ~~$[q_2]$~~

2 Equivalent states $[q_0, q_4]$ $[q_6]$ $[q_1, q_7]$ $[q_5]$ ~~$[q_2]$~~

3 Equivalent states $[q_0, q_4]$ $[q_6]$ $[q_1, q_7]$ $[q_5]$ ~~$[q_2]$~~ } same as stop here



The Transition table will be

	0	1
→ q_0	q_1	q_5
q_1	q_6	* q_2
* q_2	q_0	* q_2
q_3	q_2	q_6 X
q_4	q_7	q_5
q_5	q_2 *	q_6
q_6	q_6	q_4
q_7	q_6	* q_2

L-43 (GAT)

Minimise the

0 Equivalent

1 Equivalent

L-44

Minimise the

The Transition

	a
q_0	q_1
q_1	q_0
q_2	* q_3
* q_3	* q_3

X	q_4	q_3
Not	q_5	q_6
Reachable	q_6	q_5
	q_7	q_6

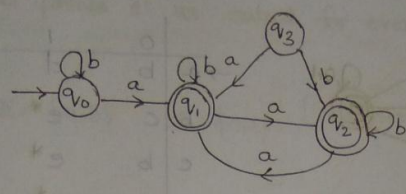
20

L-43 (GATE)

Minimise the DFA

table will be

- 1
- q_5
- $*q_2$
- $*q_2$
- q_6 X
- q_5
- q_6
- q_4
- $*q_2$



The Transition table is

	a	b
$\rightarrow q_0$	q_1^*	q_0
$*q_1$	q_2^*	q_1^*
$*q_2$	q_1^*	q_2^*

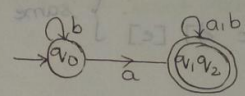
0 Equivalent states

$[q_0]$ $[q_1, q_2]$

1 Equivalent states

$[q_0]$ $[q_1, q_2]$

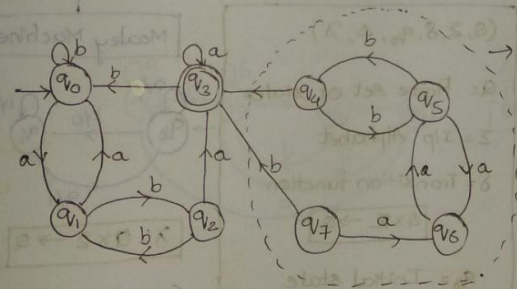
same, stop here



L-44

Minimise the DFA

same, stop here



NOT REACHABLE FROM INITIAL STATE

The Transition-table is

	a	b
q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3^*	q_1
$*q_3$	q_3^*	q_0

0 Equivalent $[q_0, q_1, q_2]$ $[q_3]$

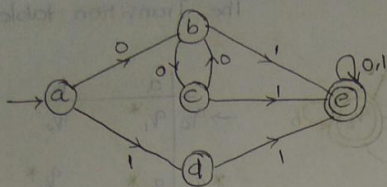
1 Equivalent $[q_0, q_1]$ $[q_2]$ $[q_3]$

2 Equivalent $[q_0]$ $[q_1]$ $[q_2]$ $[q_3]$

X	q_4	q_3	q_5
	q_5	q_6	q_4
	q_6	q_5	q_6
	q_7	q_6	q_3

L-45

Minimise the DFA



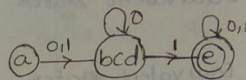
The Transition table is.

	0	1
a	b	d
b	c	e*
c	b	e*
d	c	e*
e	e	e*

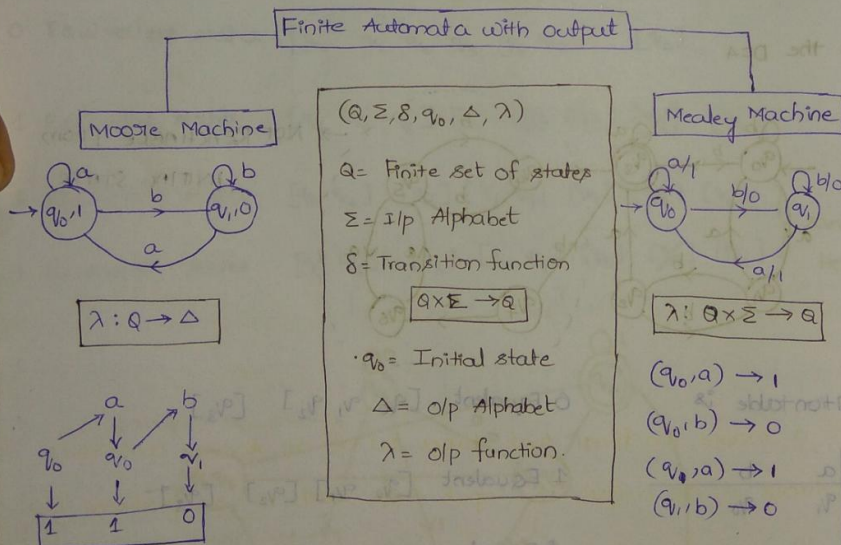
0 Equivalent states [a b c d] [e]

1 Equivalent states [a] [b c d] [e]

2 Equivalent states [a] [b c d] [e] } same



L-46 MOORE AND MEALEY MACHINES



If we give 'n' length Input then Output will be 'n' length String in Mealey Machine

L-47

Construct a moore as input and poi substring.

$\Sigma = \{a, b\}$

L-48

counting the occ

$\Sigma = \{a, b\}$

L-49

Construct a moore and produces output if %p

$\Sigma = \{0, 1\}$

10 - A

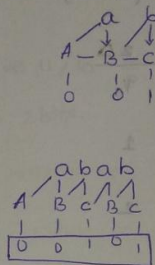
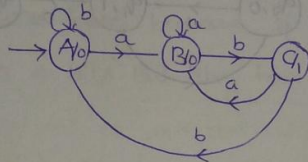
11 - B

22

L-47

construct a moore machine that takes set of all strings over $\{a,b\}$ as input and prints '1' as output for every occurrence of 'ab' as substring.

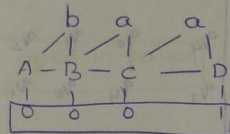
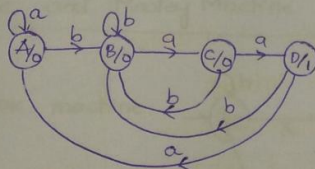
$\Sigma = \{a,b\}$ $\Delta = \{0,1\}$ [$\therefore$ construct DFA ending with ab]



L-48

counting the occurrence of substring 'baa', construct Moore Machine?

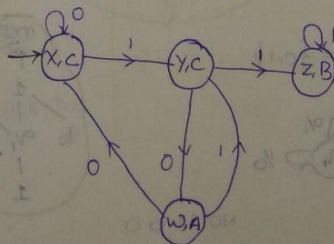
$\Sigma = \{a,b\}$ $\Delta = \{0,1\}$ [$\therefore$ construct DFA ending with 'baa']



L-49

Construct a moore machine that takes set of all strings over $\{0,1\}$ and produces 'A' as output if input ends with '10' or produces 'B' as output if i/p ends with '11' otherwise produces 'C'.

$\Sigma = \{0,1\}$ $\Delta = \{A, B, C\}$



10 - A
11 - B

h Input then
n' length String

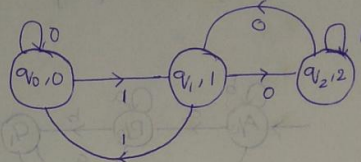
L-50

Moore Machine Residue modulo 'n'

construct a moore machine that takes binary no's as input and produces residue modulo '3' as output.

$$\Sigma = \{0, 1\} \quad \Delta = \{0, 1, 2\}$$

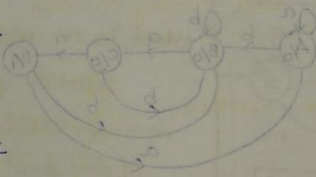
	0	1		Δ
q_0	q_0	q_1		0
q_1	q_2	q_0		1
q_2	q_1	q_2		2



construct a moore machine that takes base 4 no's as input and produces residue modulo '5' as output.

$$\Sigma = \{0, 1, 2, 3\} \quad \Delta = \{0, 1, 2, 3, 4\}$$

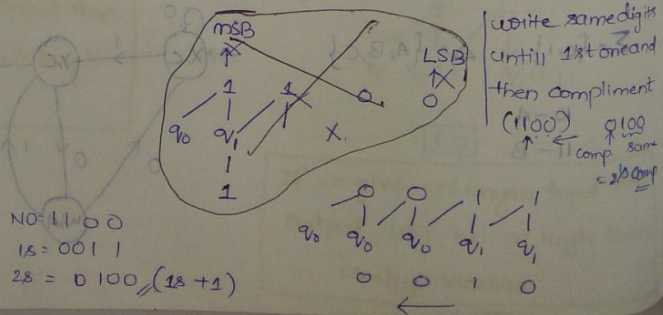
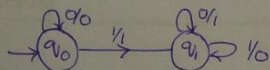
	0	1	2	3		Δ
q_0	q_0	q_1	q_2	q_3		0
q_1	q_4	q_0	q_1	q_2		1
q_2	q_3	q_4	q_0	q_1		2
q_3	q_2	q_3	q_4	q_0		3
q_4	q_1	q_2	q_3	q_4		4



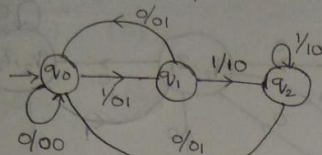
L-51 MEALY MACHINE

construct a mealy machine that takes Binary number as i/p and produces 2's complement of that no. as o/p. Assume the string is Read from Least Significant bit to most significant bit and end carry is discarded?

$$\Sigma = \{0, 1\} \quad \Delta = \{0, 1\}$$



What is the output produced by the following state machine

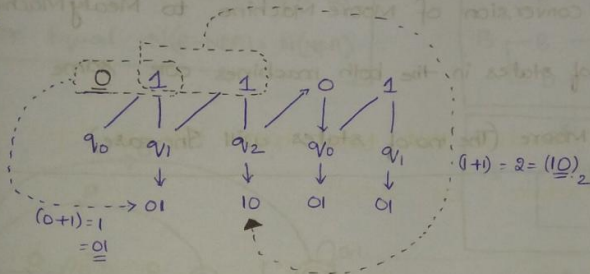

$$x a y \vdash \rightarrow 01$$
$$x_b > 10 - 00$$

c) sum of present and previous bits

$\phi > \text{NOT}$

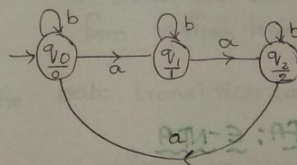
option a, b are false because if we give 11/10 as input the output must contain 4 bits. But they have given 2 bits.

option c:



CONVERSION OF MOORE MACHINE TO MEALY MACHINE

Moore Machine and Mealy Machine are equal in power.

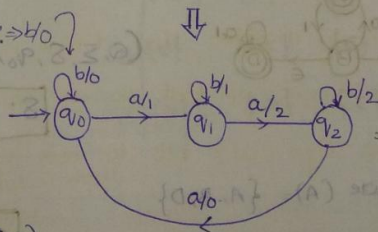


Convert the moose machine

⇒ Now, in the moore machine

(q_0, b) is going to q_0 and the

output associated with q_0 is '0' $\Rightarrow$ 4/0 $\downarrow$



⇒ Now, q_0 on 'a' is going to

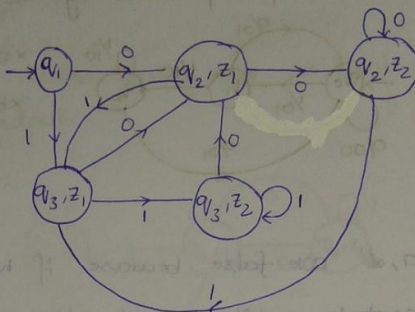
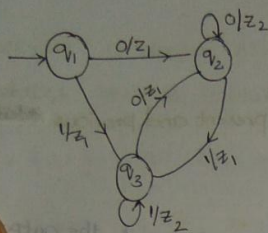
 q_i and o/p associated at q_i

is 1 $\therefore a/1$ (in Mealy machine).

→ Similarly continue for other states too.

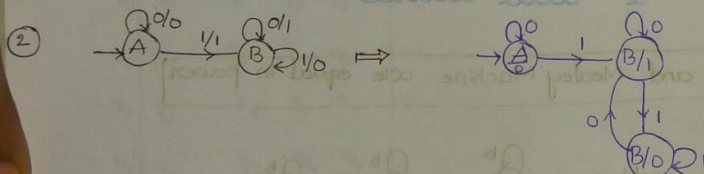
1/p and
bring is
end copy is
bite some digits
until 1st one and
then complement
(100) ← 100
↑ comp 8comp
= 2/8 comp

L-55 CONVERSION OF MEALY MACHINE TO MOORE MACHINE

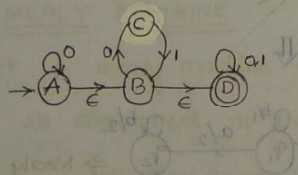


Note : 1. In the conversion of Moore Machine to Mealy Machine the No. of states in the both machines are same
2. Mealy $\rightarrow$ Moore (The no of states will Increase)

L-56



L-57 EPSILON NFA: ϵ -NFA



$(Q, \Sigma, \delta, q_0, F)$

$\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$

Now, ϵ -closure $(A) = \{A, B, D\}$
 ϵ -closure $(B) = \{B, D\}$
 ϵ -closure $(C) = \{C\}$
 ϵ -closure $(D) = \{D\}$

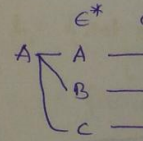
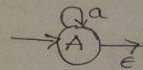
ϵ -closure (A) = The states that are Reachable from 'A' on seeing Epsilon (ϵ)

The state transition

	0
$\rightarrow A$	{ABCD}
B	{CD}
C	{ $\emptyset$ }
*D	{D}

The No. of states in NFA are 4

L-58



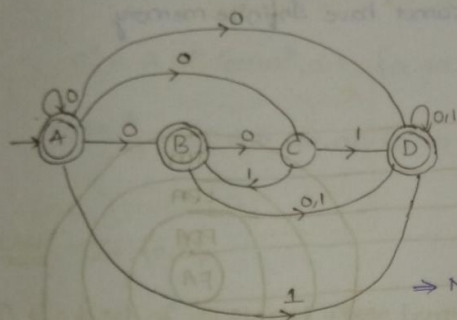
ϵ^* b
A $\rightarrow$ A
B
C
 ϵ^* b
A $\rightarrow$ A
B
C
 ϵ^* b
A $\rightarrow$ b = {B}

(46)

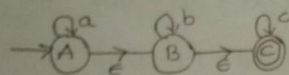
The state transition table

	0	1
→ A	{ABCD}	{D}
B	{CD}	{D}
C	{ ϕ }	{B, D}
*D	{D}	{D}

The No. of states in the E-NFA to NFA are Equal $N(E-NFA) = N(NFA)$

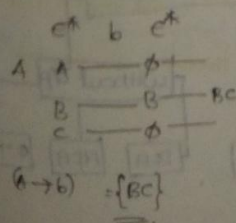
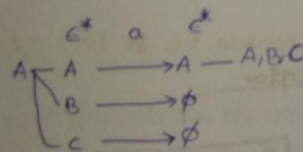


L-58

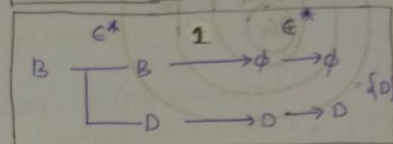
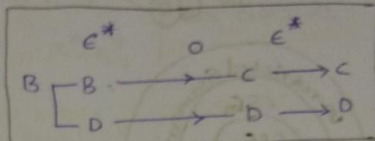
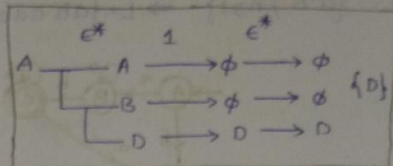
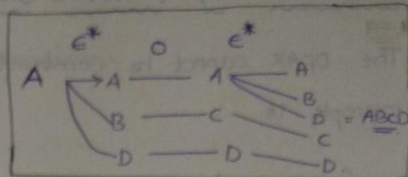


The state transition table is

	a	b	c
A	{ABC}	{BC}	{C}
B	{ ϕ }	{BC}	{C}
C	{ ϕ }	{ ϕ }	{C}



The state transition table is

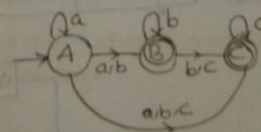


Conversion of E-NFA to NFA

→ No. of final states will increase from E-NFA to NFA conversion. (The states on which you can reach final states from epsilon transition are also final states)

The state transition table is

	a	b	c
A	{ABC}	{BC}	{C}
B	{ ϕ }	{BC}	{C}
C	{ ϕ }	{ ϕ }	{C}



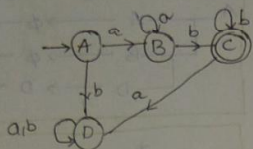
states that are on seeing

FAMILIES OF FORMAL LANGUAGES

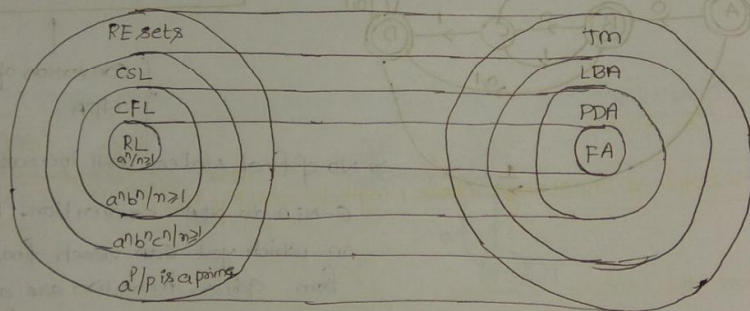
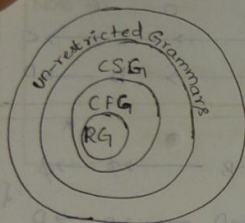
L-59

The DFA cannot be constructed for all the Languages and one such example is

$$L = \{a^n b^n / n \geq 1\} \Rightarrow L = \{ab, aabb, aaabbb, \dots\}$$

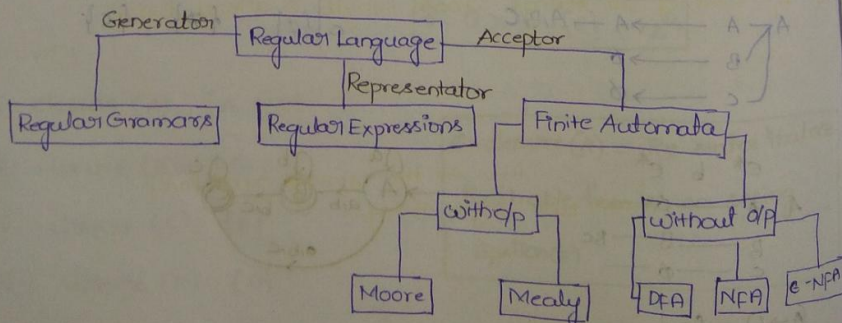


The Trap in this DFA is that the string "aab" will also be accepted, so we are unable to count the nos of 'a' arrived before 'b' so the memory required at every state will be Infinite. But FA cannot have Infinite memory



REGULAR EXPRESSIONS AND CONVERSIONS

L-60



Regular Expression

The Regular Expression

- 1) Union (+)
- 2) Concatenation (.)
- 3) Kleen closure (*)

$$\begin{aligned} a) \quad \emptyset &= \{ \} \\ \epsilon &= \{ \epsilon \} \\ a &= \{ a \} \end{aligned}$$

Now, $a^* = \{ \epsilon, a, aa, \dots \}$

$$a^+ = a a^*$$

$$(a+b)^* =$$

$$L-61 \quad \Sigma = \{ a, b \}$$

$$① L_1 = \{ \text{set of strings} \}$$

$$= \{ aa, aab, \dots \}$$

$$= \{ a^* a \}$$

$$= \{ a^+ \}$$

$$= \{ a^+ \}$$

$$② L_1 = \{ \text{set of strings} \}$$

$$= \{ aa, aab, \dots \}$$

$$= \{ a^+ \}$$

$$\{ a^+ (a+b)^* a^+ \}$$

Regular Expressions

The Regular Expression contains three operations

- 1) Union (+)
- 2) Concatenation (.)
- 3) Kleen closure (*)

$$\left. \begin{array}{l} a) \emptyset = \{ \} \\ \epsilon = \{ \epsilon \} \\ a = \{ a \} \end{array} \right\} \rightarrow \text{primitive RE's.}$$

$$\left. \begin{array}{l} (b) r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^*, r_2^* \end{array} \right\} \text{ are also RE}$$

Now, $a^* = \{ \epsilon, a, aa, aaa, \dots \}$

$a^+ = a \cdot a^* \text{ (or) } a^* \cdot a = \{ a, aa, aaa, aaaa, \dots \}$ (No epsilon)

$(a+b)^* = \{ \epsilon, a, b, aa, ab, ba, bb, \dots \} = \{ \text{set of all strings possible over } a, b \}$

L-6 $\Sigma = \{ a, b \}$

① $L_1 = \{ \text{set of all strings whose length is exactly two} \}$

$= \{ aa, ab, ba, bb \}$

$= \{ aa + ab + ba + bb \}$

$= \{ a(a+b) + b(a+b) \}$

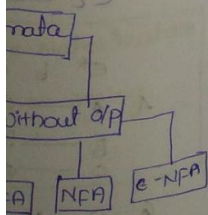
$= \{ (a+b)(a+b) \}$

② $L_1 = \{ \text{set of all strings whose length is atleast two} \}$

$= \{ aa, ab, ba, bb, aaa, bbbb, \dots \}$

$= \{ (a+b)(a+b)(a+b)^* \}$ (or) $\{ (a+b)^* (a+b)(a+b) \}$ (or) $\{ (a+b)(a+b)^* (a+b) \}$

$\hookrightarrow$ All the strings over $\{ a, b \}$.
 $\hookrightarrow$ All two Length strings



③ $L_1 = \{ \text{set of all strings whose length is atleast 2} \}$

$$L_1 = \{ \epsilon, a, b, ab, ba, bb, aa \}$$

$$L_1 = \{ \epsilon + a + b + ab + ba + bb + aa \}$$

$$L_1 = \{ (a+b+\epsilon)(a+b+\epsilon) \}$$

④ $L_1 = \{ \text{even length strings} \}$

$$= \{ \epsilon, aa, ab, ba, bb, \dots \}$$

$$= \{ (a+b)(a+b)^* \}$$

↳ Even length strings repeated any no. of times gives strings of even length.

Analysis: $((a+b)(a+b))^*$

$$= ((a+b)^2)^*$$

$$= (a+b)^{2*} = \text{even.}$$

⑤ $L_1 = \{ \text{odd length strings} \}$

$$= \{ \epsilon, aaa, a, b, aab, baa, bbb, \dots \}$$

$$= \{ ((a+b)(a+b))^* (a+b) \}$$

Analysis: $(a+b)^{2n+1}$

$$= (a+b)^{2n} (a+b)$$

$$= ((a+b)(a+b))^* (a+b)$$

⑥ $L_1 = \{ \text{Length is divisible by 3} \}$

$$L = \{ \text{'0' length strings, 3 Ls, 6 Ls, 9 Ls, \dots} \}$$

$$= \{ ((a+b)(a+b)(a+b))^* \}$$

Analysis: $(a+b)^{3n+2}$

$$= (a+b)^{3n} (a+b)^2$$

$$= ((a+b)(a+b)(a+b))^* (a+b)(a+b)$$

⑦ $L = \{ \text{Length} \equiv 2 \pmod{3} \}$

$$= \{ ((a+b)(a+b)(a+b))^* (a+b)(a+b) \}$$

⑧ $L = \{ \text{exactly 2 a's} \} \quad \{ \text{atleast 2} \} \quad \{ \text{atmost 2} \}$

$$= \{ b^* a b^* a b^* \}$$

$$\{ b^* a b^* a (a+b)^* \}$$

$$\{ b^* (\epsilon+a) b^* (\epsilon+a) b^* \}$$

⑨ $L = \{ \text{a's are even} \}$

$$= (b^* a b^* a b^*)^*$$

⑩ $L = \{ \text{starts with } a \}$

$$= \{ a(a+b)^* \}$$

⑪ $L = \{ \text{start and end with } a \}$

$$= \{ a(a+b)^* b \}$$

$$\{ b(a+b)^* a \}$$

L-62

⑫ $L = \{ \text{no 2 a's show} \}$

$$= \{ \epsilon, b, bb, bbb, \dots \}$$

$$\{ ba, bab, \dots \}$$

$$= (b+ab)^* (b+\epsilon)$$

$$(a+\epsilon)(b+ab)^*$$

⑬ $L = \{ \text{NO-2 a's} \}$

$$= \{ \epsilon, a, b, ab, \dots \}$$

Starts

$$\{ a b a b a b a \dots \} - a$$

$$\{ a b, a b a b, a b a b a b - a \}$$

$$\{ b, b a b a \dots \} - b$$

$$\{ b, b a b, b a b a b \dots \} - b$$

$$⑨ L = \{a's \text{ come even}\} = \{aa, aab, abb, bbb, \dots\}$$

$$= (b^*ab^*ab^*)^* + (b^*) \Rightarrow \text{The Language/RE with not generate the strings}$$

$\{b, bb, bbb, \dots\}$ so we add b^* to generate the strings

(31)

$$⑩ L = \{\text{starts with } a\}$$

$$\{\text{Ends with } a\}$$

$$\{\text{contains } a\}$$

$$= \{a(a+b)^*\}$$

$$\{(a+b)^*a\}$$

$$\{(a+b)^*a(a+b)^*\}$$

$$⑪ L = \{\text{start and end with diff symbols}\}$$

$$\{\text{start and end with same symbols}\}$$

$$= \{a(a+b)^*b\}$$

$$L = \{\epsilon, a, b, aa, bbb, aaa, aba, \dots\}$$

$$\{b(a+b)^*a\}$$

$$= \{a(a+b)^*a + b(a+b)^*b + a + b + \epsilon\}$$

L-62

$$⑫ L = \{\text{no 2a's should come together}\}$$

$$= \{\epsilon, b, bb, bbb, \dots, a, ab, aba, abab, ababa, \dots\}$$

$$\{ba, bab, baba, babab, \dots\} = (b+ab)^* + (b+ab)^*a \rightarrow \{\text{set of all strings ending with } a\}$$

$$= (b+ab)^*(\epsilon+a) \quad \text{or}$$

$\downarrow$
set of all strings ending with 'b'

$$(a+\epsilon)(b+ba)^*$$

$$⑬ L = \{\text{NO 2a's and no 2b's come together}\}$$

$$= \{\epsilon, a, b, ab, ba, aba, bab, \dots\}$$

Starts withEnds with

$$\{a, \underline{ba}, \underline{baba}, \dots\} - a$$

$$a - (ab)^*a \quad \text{or} \quad a(ba)^*$$

$$\{b, \underline{ab}, \underline{abab}, \underline{ababab}, \dots\} - b$$

$$b - (ab)^*b \quad \text{or} \quad a(ba)^*b$$

$$\{ba, baba, \dots\} - b$$

$$a - (ba)^* \quad \text{or} \quad b(ab)^*a$$

$$\{b, \underline{ba}, \underline{ba}, \underline{ba}, \dots\} - b$$

$$b - (ba)^*b \quad \text{or} \quad b(ab)^*$$

$$\Rightarrow (ab)^*a + (ab)^* + b(ab)^*a + b(ab)^*$$

$$= (ab)^*(a+\epsilon) + b(ab)^*(a+\epsilon)$$

$$= (ab)^*(a+\epsilon)(b+\epsilon)$$

Identities of Regular Expressions V.V.V. Imp.

32

① $\phi + R = R + \phi = R$

⑥ $\epsilon + RR^* = RR^* + \epsilon = R^*$

② $\phi \cdot R = R \cdot \phi = \phi$

⑦ $(a+b)^* = (a^* + b^*)^* = (a^*b^*)^*$
 $= (a^* + b)^*$

③ $\epsilon R = R \epsilon = R$

④ $\epsilon^* = \epsilon$

$= (a+b)^* = a^*(ba^*)^* = b^*(ab^*)^*$

⑤ $\phi^* = \epsilon$

L-63

Regular Expression to finite Automata

1> ϕ :

⑤ a^* :

2> a :

3> $a+b$:

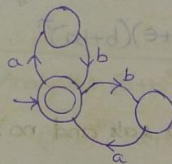
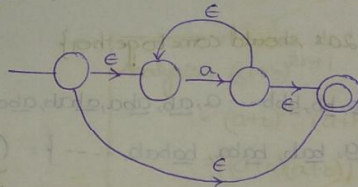
4> ab :

① ab^* :

② $(ab)^*$:

③ $(ab+ba)^*$:

STATE CREATION
METHOD

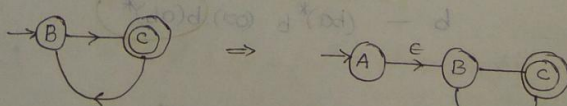


Finite Automata to Regular Expression conversion

Rule 1

The initial stage should not have any incoming edge

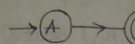
STATE ELIMINATION
METHOD



$(a+ab)^* + (a+ab)^*a^* = (a+ab)^*a^*$
 $(a+ab)^*a^* = (a+ab)^*a^*$

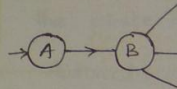
Rule 2

The final state

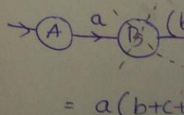
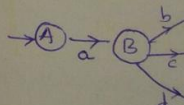
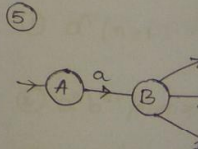
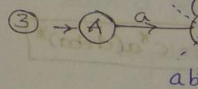
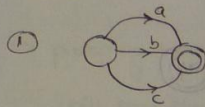


Rule 3

There must be except the

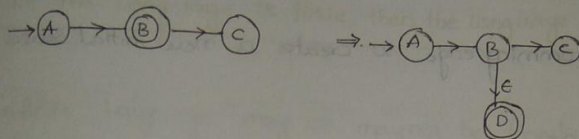


Examples



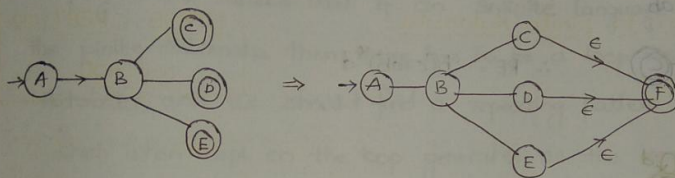
$= a(b+c)^*$

The final state should not have any outgoing edge and there should be only one final state

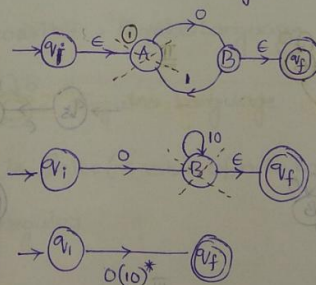
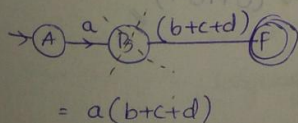
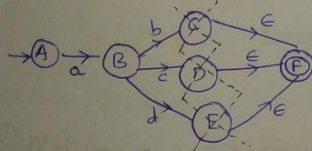
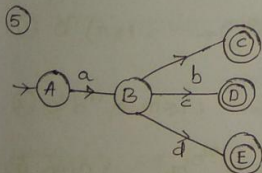
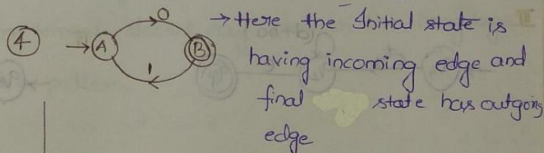
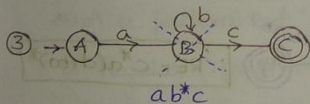
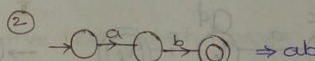
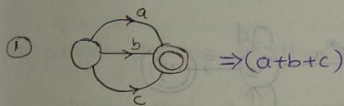


Rule-3

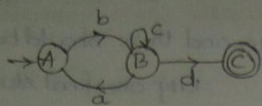
There must be only one final state and now eliminate the other states except the initial and final states in any order you wish.



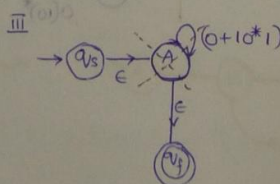
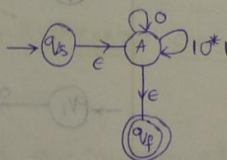
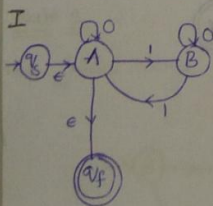
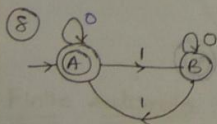
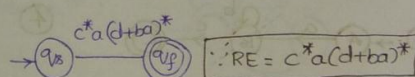
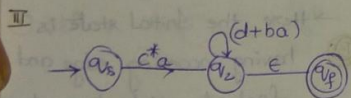
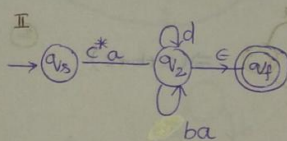
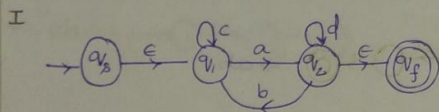
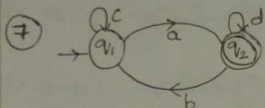
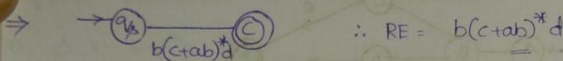
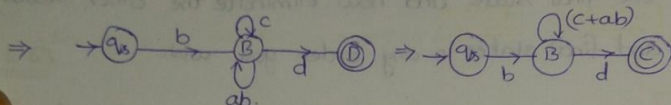
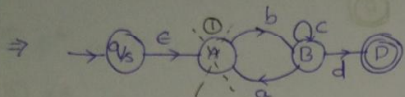
Examples



⑥



The initial state has incoming edge so create a new initial state



$$\Rightarrow (0+10^*1)^*$$

34

TESTING

L-64

If the

$\Rightarrow$ Infinite

that is

not is

$\Rightarrow$ Pumping Lemma

the finite

Automata

which w

the language

be cons

$\Rightarrow$ pumping Lemma

will say th

such a p

is Regular

① $a^n / n \geq 1$

② $a^n b^m / n, m$

③ $a^n b^n / n \leq$

④ $a^n b^n / n \geq 1$

⑤ $ww^R / |w| = 2$

TESTING WHETHER A "LANG" IS "REGULAR" OR NOT

35

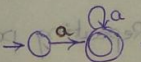
L-64

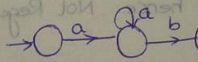
If the Language is finite then the language is Regular

⇒ Infinite Language may or may not be Regular. And the Algorithm that is used to find whether the given language is Regular or not is called "PUMPING LEMMA".

⇒ Pumping Lemma states that if an Infinite language has to be accepted by the finite Automata then there has to be a loop inside the finite Automata, and we should find a repeating pattern in the Language which when kept on the loop generates all the remaining strings in the Language, if we do not find any such pattern then No DFA can be constructed for that Language and the language is not Regular.

⇒ Pumping Lemma is the Negative test which means Pumping Lemma will say that a language is not Regular if we are not able to find such a pattern but Pumping Lemma doesn't say whether the language is Regular if you find such a pattern.

① $a^n / n \geq 1$:  → DFA possible = Regular Language

② $a^n b^m / n, m \geq 1$:  = Regular Language

③ $a^n b^n / n \leq 10^{10} 10^{10}$: The Language is finite ($\because$ value of n is bounded), so the Language is Regular

④ $a^n b^n / n \geq 1$: Not Regular because we need to count the Infinite occurrence of 'a's before 'b' but FA has finite memory so the language is not Regular.

⑤ $w w^R / |w| = 2, w \in \{a, b\}$: $\left\{ \begin{array}{l} aaaa \\ abba \\ baab \\ bbbb \end{array} \right\}$ - Finite Language $\Rightarrow$ Regular language

⑥ $w^R/w \in \{a,b\}^* = \{\text{set of all palindromes}\} = \text{The Language is not Regular}$

because the string 'w' can be of any length and that must be compared against w^R and the word/string 'w' can be infinite length, hence FA has no capacity to store infinite length strings $\Rightarrow$ Not- Regular Language

⑦ $a^n b^m c^k / n, m, k \geq 1 = \text{Regular Language [No comparisons, a,b,c generated independently]}$

$\rightarrow aa^* bb^* cc^*$

⑧ $a^i b^j / i, j \geq 1 = \text{Regular Language} = aa^* bb(bb)^*$

Now, $\Sigma = \{a\}$

⑨ $a^n / n \text{ is even} = \{a^0, a^2, a^4, a^6, a^8, \dots\}$ difference can be kept in loop of DFA

$a^n / n \text{ is odd} = \{a^1, a^3, a^5, a^7, \dots\} = \text{Regular Language}$

⑩ $a^n / n \text{ is prime} = \{a^1, a^3, a^5, a^7, a^{11}, \dots\} = \{\text{Not in } \mathbb{N} \Rightarrow \text{No Repeating pattern}\}$

$a^{n^2} / n \geq 1 = \{a^1, a^4, a^9, a^{16}, a^{25}, \dots\} = \{\text{Not in } \mathbb{N} \Rightarrow \text{Not RL}\}$

$a^{n^2} a^{n^3} / n \geq 1 = \{a^1, a^2, a^4, a^8, a^{16}, \dots\} = \{\text{Not in } \mathbb{N} \Rightarrow \text{Not RL}\}$

$\Sigma = \{a, b\}$

⑪ $a^i b^j / i, j \geq 1 = \text{Even the } a, b \text{ are generated independently, due to } j^2 \text{ there is not Repeating pattern in the Language and hence Not Regular}$

⑫ $a^i b^{2n} / i, n \geq 1 = a^i \text{ can be independently generated and } 2n/2 \text{ to the power } n \text{ has no Repeating pattern.}$

⑬ $a^i b^p / i \geq 1 = b^p \text{ cannot be generated and No Repeating pattern.}$

⑭ $\{w / n_a(w) = n_b(w)\} = \text{Not Regular Language (counting a's need Infinite Memory)}$

$$n_a(w) \leq n_b(w)$$

$$n_a(w) \geq n_b(w)$$

⑮ $n_a(w) \bmod 3 \leq$

⑯ $a^n / n \geq 10 \Rightarrow$

⑰ $ww^R / w \in \{a,b\}^*$

⑱ $a^n b^n c^n / n \geq 1$

⑲ $a^n b^{n+m} c^m / n, m \geq 1$

Imp V.V. Imp

⑳ $wxw^R / w, x \in \{a,b\}^*$

㉑ $wxw^R / w \in \{0,1\}^*$
 $|x| = 5$

㉒ $xww^Ry / x, y \in \{a,b\}^*$

㉓ $xww^R / x, w \in \{a,b\}^*$

㉔ $ww^Ry / w, y \in \{a,b\}^*$

㉕ Set of all s

15) $n_a(w) \bmod 3 \leq n_b(w) \bmod 3 \Rightarrow$ Regular Language (DFA can be constructed for mod 3 counters) (37)

16) $a^n / n \geq 10 \Rightarrow$ Regular Language (DFA can be constructed)

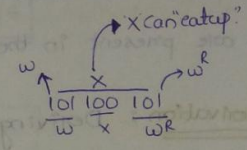
17) $ww^R / w \in (a,b)^* =$ Not Regular Language

18) $a^n b^n c^n / n \geq 1 =$ Not Regular Language

19) $a^n b^{n+m} c^m / n, m \geq 1 =$ Not Regular Language

Imp V.V Imp

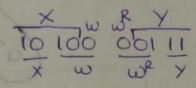
20) $wxw^R / w, x \in (0,1)^+ : \text{Regular Language}$
 $w = 101$
 $x = 100$
 $w^R = 101$



$$RE = 1(0+1)^*1 + 0(0+1)^*0$$

21) $wxw^R / w \in (0,1)^+ : \text{Not Regular Language}$ ($\because$ 'x' cannot go beyond 'w' length)
 $|x| = 5$

22) $xww^Ry / x, y, w \in (0,1)^+ :$
 $x = 10$
 $y = 11$
 $w = 100$
 $w^R = 001$



$\therefore$ Regular Language

$$RE: (0+1)^* 00 (0+1)^* \\ (or) \\ (0+1)^* 11 (0+1)^*$$

23) $xww^R / x, w \in (0,1)^+ : \text{Not Regular Language}$

24) $ww^Ry / w, y \in (0,1)^+ : \text{Not Regular Language}$

25) Set of all strings having equal number of '01' and '10' is Regular.
 $0(0+1)^*0 + 1(0+1)^*1 + 1+0$

GRAMMARS

L-65

A Grammar is generally represented by (V, T, P, S)

V = vertices
 T = Terminals
 P = productions
 S = start symbol

$S \rightarrow aSB$ $V = \{S, B\}$ $P = \left\{ \begin{array}{l} S \rightarrow aSB \\ S \rightarrow aB \\ B \rightarrow b \end{array} \right\}$ $S = \{S\}$
 $S \rightarrow aB$
 $B \rightarrow b$

→ The main intention of the grammar is to generate all the strings that are present in the language

Derivation: Deriving the string from the grammar using the start symbol

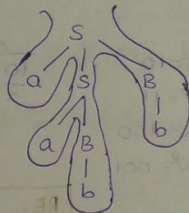
$S \rightarrow aSB$
 $S \rightarrow aB$
 $B \rightarrow b$

Derivation of aabb

$S \rightarrow aSB$
 $\Rightarrow a a B B$
 $\Rightarrow a a b B$
 $\Rightarrow a a b b$

Sentential forms (or) Sequential forms.

Left most derivation



CONSTRUCTION OF GRAMMARS

① $L = \{aa, ab, ba, bb\}$ $S \rightarrow aa/ab/ba/bb$
 $(a+b) \cdot (a+b)$
 $A \quad A$

$S \rightarrow AA$
 $A \rightarrow a/b$

② $L = \{a^n/n \geq 1\}$ - $L = \{a, aa, aaa, aaaa, \dots\}$

$A \rightarrow aA/\epsilon$ (or)

$S \rightarrow A$
 $A \rightarrow aA/a$

③ $L = (a+b)^*$ $L = \{\epsilon, a, b, ab, ba, aa, bb, \dots\}$

$S \rightarrow aS/bS/\epsilon$

④ $L = \{\text{set of all}\}$
 $= (a+b)(a+b)$

⑤ $L = \{\text{Length}\}$
 $= (a+b+\epsilon)$
 A

⑥ $L = \{\text{start w}\}$
 $= a(a+b)$

⑦ $L = \{\text{start and e}\}$
 $= a(a+b)^*$

$L = \{\text{same}\}$

⑧ $L = \{a^n b^n/n \geq 1\}$

⑨ $L = \{waw^R\} \cup \{waw\}$
 $w \in (a+b)^*$

⑩ $L = \{\text{set of a}\}$
 $(a+)$
 A

⑪ $L = \{a^n b^m/n, m\}$

⑫ $L = \{a^n c^m b^n\}$

39

④ $L = \{ \text{set of all strings of length at least 2} \}$

$$= (a+b)(a+b)(a+b)^*$$

$$S \rightarrow AAB$$

$$A \rightarrow a/b$$

$$B \rightarrow aB/bB/\epsilon$$

39

⑤ $L = \{ \text{Length at most 2} \}$

$$= (a+b+\epsilon)(a+b+\epsilon)$$

$$S \rightarrow AA$$

$$A \rightarrow a/b/\epsilon$$

⑥ $L = \{ \text{start with 'a' end with 'b'} \}$

$$= a(a+b)^*b$$

$$S \rightarrow aAb$$

$$A \rightarrow aA/bA/\epsilon$$

⑦ $L = \{ \text{start and end with diff symbols} \}$

$$= a(a+b)^*b + b(a+b)^*a$$

$$S \rightarrow aAb/bAa$$

$$A \rightarrow aA/bA/\epsilon$$

⑧ $L = \{ \text{same symbols} \}$

$$S \rightarrow aAa/bAb/a/b/\epsilon$$

$$A \rightarrow aA/bA/\epsilon$$

⑨ $L = \{ a^n b^m / n, m \geq 1 \}$

$$S \rightarrow aSb/ab$$

⑩ $L = \{ wuw^k \mid u \in \{a,b\}^+, w \in \{a,b\}^+, k \geq 1 \}$

$$S \rightarrow aSa/bSb/a/b/\epsilon$$

$$w \in (a+b)^*$$

⑪ $L = \{ \text{set of all strings of even length} \}$

$$\frac{(a+b)(a+b)^*}{A \quad A}$$

$$S \rightarrow BS/\epsilon$$

$$B \rightarrow AA$$

$$A \rightarrow a/b$$

⑫ $L = \{ a^n b^m / n, m \geq 1 \}$

$$S \rightarrow AB$$

$$A \rightarrow aA/a$$

$$B \rightarrow aB/b$$

⑬ $L = \{ a^n b^m c^k / n, m, k \geq 1 \}$

$$S \rightarrow ABC$$

$$A \rightarrow aAb/ab$$

$$B \rightarrow cB/c$$

⑭ $L = \{ a^n c^m b^n / n, m \geq 1 \}$

$$S \rightarrow aSb/aAb$$

$$A \rightarrow cA/c$$

PUSH DOWN AUTOMATA

L-68

Push Down Automata = Finite Automata + stack

Push Down Automata = $(Q, \Sigma, \delta, q_0, Z_0, F, \Gamma)$

Q = finite set of states

Z_0 = Bottom of the stack

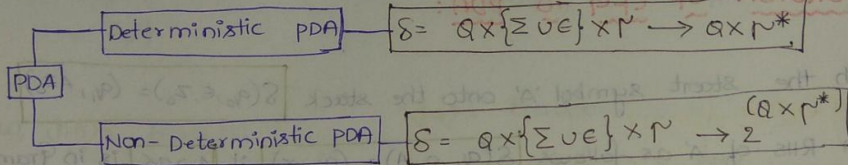
Σ = Input symbol

F = set of final states

δ = Transition function

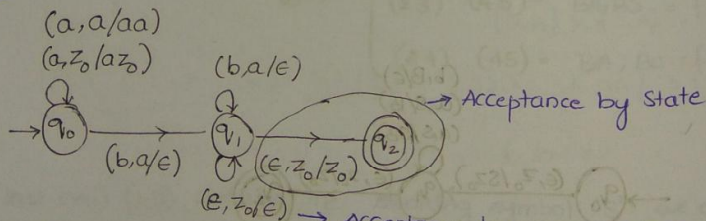
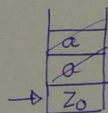
Γ = stack Alphabet

q_0 = Initial state



① $L = \{a^n b^n / n \geq 1\}$

Let the string be aabb
↑↑↑↑



$\delta(q_0, a, z_0) = (q_0, az_0)$ → Acceptance by Empty stack

$\delta(q_0, a, a) = (q_0, aa)$

$\delta(q_0, b, a) = (q_1, \epsilon)$

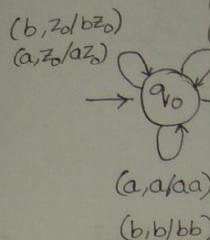
$\delta(q_0, b, b) = (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$ (or) (q_1, ϵ)

Acceptance by state

→ Acceptance by Empty stack

② $w \in (arb)/n_0$

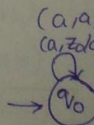


③ $L = \{a^n b^n c^n\}$

L-69

④ $L = \{a^n b^m c^n\}$

⑤ $L = \{a^{m+n} b\}$

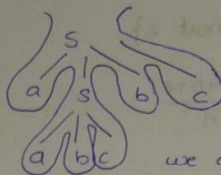


$$(13) L = \{a^n b^n c^m d^m / n, m \geq 1\}$$

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aAb/ab \\ B &\rightarrow cBd/cd \end{aligned}$$

$$(14) L = \{a^n b^n c^n / n \geq 1\}$$

$$S \rightarrow aSbc/abc$$



aabcbb

$$= a^n (bc)^n$$

For this language

we cannot give such grammar & we give 'CSG' for this language

$$(15) L = \{a^n b^{2n} / n \geq 1\}$$

$$S \rightarrow aSbb/abb$$

$$(16) L = \{a^n b^m c^m d^n / n, m \geq 1\}$$

$$\begin{aligned} S &\rightarrow aSd/aAd \\ A &\rightarrow bAc/bc \end{aligned}$$

$$(17) L = \{a^n b^m c^n d^m / n, m \geq 1\} \Rightarrow \text{Not possible to give CFG.}$$

$$(18) L = \{a^n a^m b^m c^n\}$$

$$\begin{aligned} S &\rightarrow aSc/aAc \\ A &\rightarrow aAb/ab \end{aligned}$$

$$(19) L = \{a^n b^{n+m} c^m / n, m \geq 1\}$$

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aAb/ab \\ B &\rightarrow bBc/bc \end{aligned}$$

$$(20) L = \{a^n b^m c^{n+m} / n, m \geq 1\}$$

$$\begin{aligned} S &\rightarrow aSc/aAc \\ A &\rightarrow bAc/bc \end{aligned}$$

$$= \{a^n b^m c^m c^n / n, m \geq 1\}$$

L-66

CLASSIFICATION OF GRAMMARS

According to Chomsky the Grammars are classified into 4 types.

1> Type '3' / Regular Grammars.

2> Type '2'

3> Type '1'

4> Type '0'

Type -3:

If the Grammar has all the productions of the form

Right Linear
Grammar
(RLG)

$$A \rightarrow \alpha B / \beta$$

$$\begin{aligned} (A, B) &\in V \\ (\alpha, \beta) &\in T^* \end{aligned}$$

$$A \rightarrow B \alpha / \beta$$

$$\begin{aligned} (A, B) &\in V \\ (\alpha, \beta) &\in T^* \end{aligned}$$

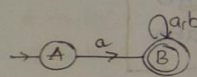
Left Linear
Grammar
(LLG)

Ex:

$$\begin{aligned} A &\rightarrow aB/a \\ B &\rightarrow aB/a \end{aligned}$$

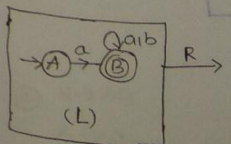
The Languages
are always

CONVERSION OF



Finite Automata

1> convert the



40

Ex:

RLG

$A \rightarrow aB/a$
 $B \rightarrow aB/bB/a/b$

LLG

$A \rightarrow Ba/a$
 $B \rightarrow Ba/bB/a/b$

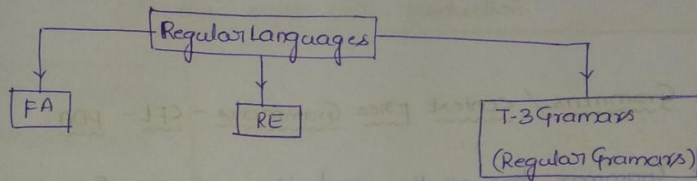
NOT T-3.

$A \rightarrow Ba/a$
 $B \rightarrow aB/a$

41

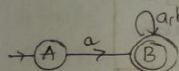
The Languages that are generated by Type-3 Grammar are always Regular Languages.

{ T-3 Grammars should be either LLG or RLG but not both. }



CONVERSION OF FINITE AUTOMATA TO REGULAR GRAMMAR

$\Rightarrow$ The Initial state in the Automata is same as the start state in the Grammar



$A \rightarrow aB$
 $B \rightarrow aB/bB/\epsilon$

RLG

$L = \{a, aa, ababab, \dots\}$
start with 'a'

$\Downarrow R$

$A \rightarrow Ba$
 $B \rightarrow Ba/bB/\epsilon$

LLG

$L = \{a, ba, babab, \dots\}$

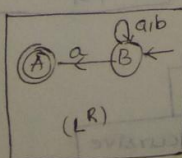
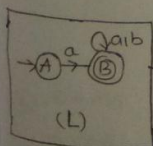
Ending with 'a'

4 types -

Finite Automata to LLG:

$FA \xrightarrow{R} FA \xrightarrow{R} RLG \xrightarrow{R} LLG$
 $(L) \rightarrow (L^R) \rightarrow (L^R) \rightarrow (L^R)^R = L$

$\Rightarrow$ convert the above FA to LLG.



$B \rightarrow aB/bB/aA$
 $A \rightarrow \epsilon$

$B \rightarrow Ba/bB/Aa$
 $A \rightarrow \epsilon$

LLG (L)

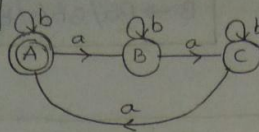
Left Linear Grammar (LLG)

CONVERSION OF REGULAR GRAMMARS TO FINITE AUTOMATA

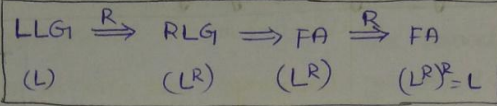
42

$A \rightarrow aB/bA/b$
 $B \rightarrow aC/bB$
 $C \rightarrow aA/bC/a$

RLG $\rightarrow$ FA



LLG $\rightarrow$ FA CONVERSION :



Type-2 Grammars / context free Grammars - CFL - PDA

If the Grammar has all the productions of the form

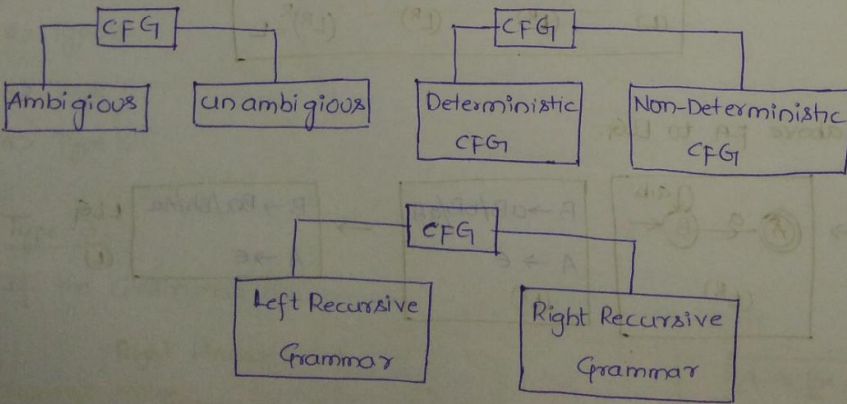
$A \rightarrow \alpha$
 $\alpha \in (V \cup T)^*$

Ex: $A \rightarrow aAb/b$
 $A \Rightarrow aAb$
 $\Rightarrow aabb$

$\Rightarrow$ The Grammar in the example is called context free Grammar because we are replacing 'A' without any content (condition) that it must be preceded by 'a' likewise.

L-67

Type 2: context free Grammars



ELIMINATION

- $S \rightarrow aSb$
 $A \rightarrow \epsilon$
- $S \rightarrow AB$
 $A \rightarrow aA$
 $B \rightarrow bB$

- $S \rightarrow Aba$
 $A \rightarrow BC$
 $B \rightarrow B/e$
 $C \rightarrow D/e$
 $D \rightarrow d$

ELIMINATION

- $S \rightarrow Aa/b$
 $B \rightarrow A/bb$
 $A \rightarrow a/bc$

- $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow C/b$
 $C \rightarrow D$
 $D \rightarrow \epsilon$

$$\textcircled{1} \begin{array}{l} S \rightarrow aSb/aAb \\ A \rightarrow \epsilon \end{array}$$

$$\Rightarrow \boxed{S \rightarrow asb/ab}$$

$$S \rightarrow asb/\cancel{asb}/ab$$

No production with ϵ

$$\Rightarrow \boxed{S \rightarrow asb|ab}$$

$$\textcircled{2} \begin{array}{l} S \rightarrow AB \\ A \rightarrow aAA/\epsilon \\ B \rightarrow bBB/\epsilon \end{array}$$

Nullable states: $\{A, B, S\}$

$\Rightarrow$ Whenever the start state is nullable then the language contains ϵ and it must be present in the start state production.

$$\boxed{\begin{array}{l} S \rightarrow AB/B/A/\epsilon \\ A \rightarrow aAA/aA/a \\ B \rightarrow bBB/bB/b \end{array}}$$

$$\textcircled{3} \begin{array}{l} S \rightarrow AbaC \\ A \rightarrow BC \\ B \rightarrow B/\epsilon \\ C \rightarrow D/\epsilon \\ D \rightarrow d \end{array}$$

Nullable symbols: $\{B, C, A\}$

$$S \rightarrow AbaC/Ab\epsilon/ba/baC$$

$$A \rightarrow BC/B/C/\epsilon$$

$$C \rightarrow D$$

$$D \rightarrow d$$

(Don't add epsilon even though B, C are nullable ($\because$ need to add it only for starting symbol)).

ELIMINATION OF UNIT PRODUCTIONS.

$$\textcircled{1} \begin{array}{l} S \rightarrow Aa/B \\ B \rightarrow A/bb \\ A \rightarrow a/bc/B \end{array}$$

Step-1:

$$\boxed{\begin{array}{l} S \rightarrow Aa/bb/a/bc \\ B \rightarrow bb/a/bc \\ A \rightarrow a/bc/bb \end{array}}$$

$$\Rightarrow S \rightarrow B \rightarrow bb$$

$$\Rightarrow S \rightarrow \cancel{A} \rightarrow A \rightarrow \begin{array}{l} a \\ bc \end{array}$$

$$\Rightarrow B \rightarrow \cancel{A} \rightarrow \begin{array}{l} a \\ bc \end{array}$$

$$\Rightarrow A \rightarrow \cancel{B} \rightarrow bb \\ A \rightarrow bb$$

$$\textcircled{2} \begin{array}{l} S \rightarrow AB \\ A \rightarrow a \\ B \rightarrow C/b \\ C \rightarrow D \\ D \rightarrow E \\ E \rightarrow a \end{array}$$

$$B \rightarrow \cancel{C} \rightarrow D \rightarrow E \rightarrow a$$

$$C \rightarrow \cancel{D} \rightarrow E \rightarrow a$$

$$D \rightarrow \cancel{E} \rightarrow a$$

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b/a$$

$$C \rightarrow a$$

$$D \rightarrow a$$

$$E \rightarrow a$$

} Useless symbols
Not Reachable
from start state

ELIMINATION OF USELESS SYMBOLS

① $S \rightarrow A\cancel{B}/a$

$A \rightarrow \cancel{B}C/b$

$B \rightarrow AB/C$

$C \rightarrow aC/B$

$T = \{a, b\}$

$V = \{S, A, B, C\}$

Now, the useful symbols (which will generate some terminals) are $\{a, b, S, A\}$

$\Rightarrow B, C$ are not useful symbols ($\because$ I could not generate any string from B, C).

$S \rightarrow$

a

$A \rightarrow b$

But here A' is not Reachable, therefore the minimised Grammar will be $\boxed{S \rightarrow a}$

② $S \rightarrow AB/\cancel{A}C$

$A \rightarrow aAb/bAa/a$

$B \rightarrow bA/aAB/AB$

$C \rightarrow abCA/aDb$

$D \rightarrow bD/aC$

$T = \{a, b\}$

$V = \{S, A, B, C, D\}$

Useful symbols = $\{a, b, A\}$

$\{B, S\}$

$S \rightarrow AB$

$A \rightarrow aAb/bAa/a$

$B \rightarrow bA/aAB/AB$

③ $S \rightarrow \cancel{ABC}/\underline{BaB}$

$A \rightarrow aA/\cancel{B}C/aaa$

$B \rightarrow bBb/a$

$C \rightarrow CA/\underline{AC}$

$T = \{a, b\}$

$V = \{S, A, B, C\}$

Useful symbols: $\{a, b, B, A, S\}$

$S \rightarrow BaB$

$A \rightarrow aA/aaa$

$B \rightarrow bBb/a$

$\rightarrow$ Not Reachable

$\Rightarrow S \rightarrow BaB$

$B \rightarrow bBb/a$

CNF - INTRODUCTION

CNF:

If all the A, B are variables

Advantages

- 1> Length of string
- 2> Derivation
- 3> The no. of symbols
- 4> It is easy to convert

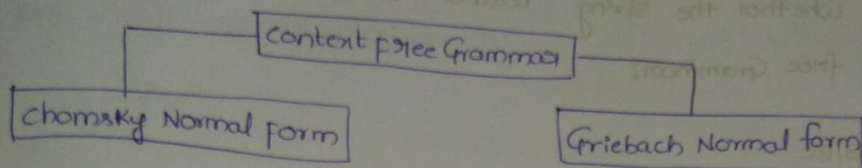
① Convert

$S \rightarrow bA/a$

$A \rightarrow bA/a$

$B \rightarrow aB$

CNF - INTRODUCTION [CHOMSKY - NORMAL FORM] 45



CNF:

If all the productions are of the form $A \rightarrow BC$ or $A \rightarrow a$ where A, B are variables and a is a terminal.

Advantages:

- 1) Length of each production is Restricted
- 2) Derivation tree (or) parse tree obtained from CNF is always Binary tree
- 3) The no. of steps required to derive a string of length $|w|$ is $(2|w|-1)$
- 4) It is easy to apply cyk membership Algorithm: $O(n^3)$ time

① Convert the following into CNF?

$$S \rightarrow bA / aB$$

$$A \rightarrow bAA / aS / a$$

$$B \rightarrow aBB / bS / b$$

Let $\begin{cases} N_a \rightarrow a \\ N_b \rightarrow b \end{cases}$ then

$$S \rightarrow N_b A / N_a B$$

$$A \rightarrow N_b \textcircled{AA} / N_a S / a$$

$$B \rightarrow N_a \textcircled{BB} / N_b S / b$$

$$N_a \rightarrow a$$

$$N_b \rightarrow b$$

$$\Rightarrow \begin{cases} S \rightarrow N_b A / N_a B \\ A \rightarrow N_b T / N_a S / a \\ B \rightarrow N_a P / N_b S / b \\ N_a \rightarrow a \\ N_b \rightarrow b \\ T \rightarrow AA \\ P \rightarrow BB \end{cases}$$

CYK ALGORITHM (MEMBERSHIP ALGORITHM)

1) Check whether the string "baaba" is a valid member of the following context free Grammar?

$$S \rightarrow AB/BC$$

$$A \rightarrow BA/a$$

$$B \rightarrow cc/b$$

$$C \rightarrow AB/a$$

Sol: The CYK Algorithm is used to check the string belong to the language generated by the Grammar or not.

⇒ The CYK Algorithm can be applicable for only CNF form Grammars

Given Grammar

$$S \rightarrow AB/BC$$

$$A \rightarrow BA/a$$

$$B \rightarrow cc/b$$

$$C \rightarrow AB/a$$

Given string

baaba
12345

	5	4	3	2	1
1	A, S, C	∅	∅	A, S	B
2	S, A, C	B	B	A, C	
3	B	S, C	A, C		
4	A, S	B			
5	A, C				

Cell no (1,1):

Now the (1,1) represent the variable 'b' in the required string
↓
[1 to 1]

⇒ Now check what are the variables that are required to generate 'b'

⇒ B ✓

So the value of cell (1,1) = B

The value

Now see

Cell no (3,3)

The Alphabet

Cell no (4,4)

(4,4) = b

Cell no (1,2)

(1,2) mean

(1,2)

Cell no (2,3)

(2,3) = (2,

1,1)

{2, A}

Cell no (3,4)

(3,3) x (4,

46

cell no (2,2):

→ start with '2' and end with '2'.

following

The value of (2,2) in the given string is 'a' (2 to 2)

Now see which variable derives 'a'

$$= \{A, C\}$$

cell no (3,3):The Alphabet in the (3,3) is 'a' so $\{A, C\}$ will generate 'a'

$$= \{A, C\}$$

cell no: (4,4)cell no (5,5):

(4,4) = b ⇒ Generated by 'B'

(5,5) = 'a' = $\{A, C\}$ will generate 'a'.cell no (1,2):

$$\begin{bmatrix} 1 & 2 \\ b & a \end{bmatrix} \begin{matrix} 3 & 4 & 5 \\ a & b & a \end{matrix}$$

(1,2) means start with '1' and end with '2' ⇒ "ba"

$$(12) = (11) \times (22) \quad [\text{cross product}]$$

$$= \{B\} \times \{A, C\} = \{BA, BC\} = \{A, S\}$$

Generated by

cell no (2,3):

$$(2,3) = (22) \times (33)$$

$$= \{A, C\} \times \{A, C\}$$

$$= \{AA, AC, CA, CC\} = \{B\}$$

$$\begin{matrix} \times & \times & \times \\ & & B \end{matrix}$$

↳ Not derived by any of the symbols.

cell no (3,4):

$$(4,5) = (44) \times (55)$$

$$(33) \times (44) = \{A, C\} \times \{B\}$$

$$= \{B\} \times \{A, C\}$$

$$\begin{matrix} \{AB, CB\} \\ SC \times \end{matrix}$$

$$= \{BA, BC\}$$

$$= \{A, S\} = \{A, S\}$$

generate

$$(1,3) = \begin{bmatrix} b & a & a \\ 1 & 2 & 3 \end{bmatrix} \begin{matrix} b & a \\ 4 & 5 \end{matrix} = \underline{(baa)}$$

$$(1) \mid (2 \ 3)$$

$$(12) \mid (3)$$

$$(1,3) = (11)(23) \quad \text{or} \quad (12)(33)$$

$$\begin{aligned} &= \{B\} \times \{B\} \quad : \quad \{A,S\} \times \{A,C\} \\ &= \{BB\} \quad : \quad \{AA, AC, SA, SC\} \\ &= \{\emptyset\} \quad : \quad \{x \times x \times x\} = \{\emptyset\} \end{aligned}$$

$$(2,4) : (22) \times (3,4) \quad \text{or} \quad (23) \times (44)$$

$$\begin{aligned} &= \{A,S\} \times \{S,C\} \quad \text{or} \quad \{B\} \times \{B\} \\ &= \{AS, AC, SS, SC\} \quad : \quad \{BB\} \\ &\quad \times \quad \times \quad \times \quad \times \quad : \quad \{\emptyset\} \\ &= \{\emptyset\} \end{aligned}$$

similarly continue for remaining cells

$$\text{Now, } (1,5) = b a a b a$$

$$(14) = b a a b$$

$$(1,5) = \begin{cases} (11) (25) = \{A,S\} \\ (12) (35) = \{S,C\} \\ (13) (45) = \{\emptyset\} \\ (14) (55) = \{\emptyset\} \end{cases} \quad (14) = \begin{cases} (11) (24) = BBX \\ (12) (34) = AS, AC, SS, SC \times \\ (13) (44) = \{\emptyset\} \end{cases}$$

$$(2,5) = \begin{matrix} 2 & 3 & 4 & 5 \end{matrix}$$

$$(2,5) = \begin{cases} (22) (35) = AB, CB \rightarrow \{S,C\} \\ (23) (45) = BA, BS = \{A\} \\ (24) (45) = BA, BC = \{A,S\} \end{cases}$$

If the cell (last cell) (1,5) contains the starting symbol 'S' we can generate the string from the given Grammar

Now, what are the various substrings that can be generated from the Grammar?

Now check where the starting symbols are present in the table, i.e. it is present at cells (1,2) (3,4) (4,5) (2,5) (1,5)

baaba
1 2 3 4 5

$$\begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ \{ba\} & \{ab\} & \{ba\} & \{aaba\} & \text{Given string} \\ & & & & = \{ba, ab, aaba\} \end{matrix}$$

② GRIE BACH

If all the $x \in V^*$ then it

$$\text{Ex: } A \rightarrow aB \\ A \rightarrow a$$

Advantages

- 1> The no. of
- 2> GNF is

CONVERSION

- 1> Push the
- 2> push RHS
- 3> Add final

$$\text{① } S \rightarrow aSb$$

$$\begin{matrix} S \rightarrow aSb \\ B \rightarrow b \end{matrix}$$

GRIE BACH NORMAL FORM (GNF)

49

If all the productions of the Grammar are of the form $A \rightarrow \alpha x$ where $x \in V^*$ then it is called GNF

Ex: $A \rightarrow aBCDE$
 $A \rightarrow a$

Advantages

- 1) The no. of steps required to generate a string of length $|w|$ is $|w|$.
- 2) GNF is useful in order to convert a CFG to "push down Automata"

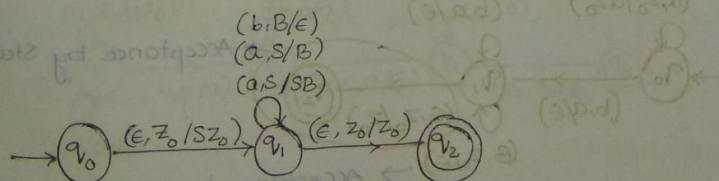
CONVERSION OF CFG TO PDA:

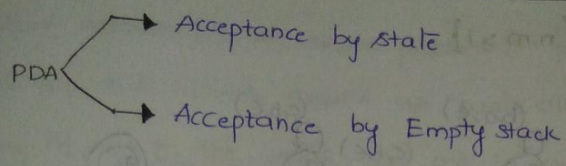
- 1) Push the start symbol 'A' onto the stack $\delta(q_0, \epsilon, Z_0) = (q_1, AZ_0)$
 - 2) push RHS of 'A' as follows $\delta(q_1, a, A) = (q_1, \alpha)$ if $A \rightarrow \alpha$ is in Grammar
 - 3) Add final state with $\delta(q_1, b, A) = (q_1, b)$ if $A \rightarrow b$ is in Grammar
- $\delta(q_1, \epsilon, Z_0) = (q_f, \epsilon)$

① $S \rightarrow aSb/ab \rightarrow$ Not in GNF

$S \rightarrow aSB/ab$
 $B \rightarrow b$

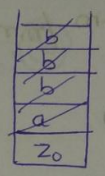
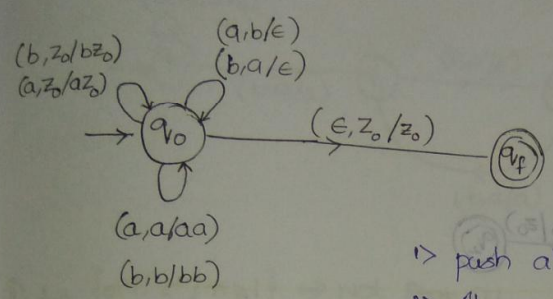
In GNF





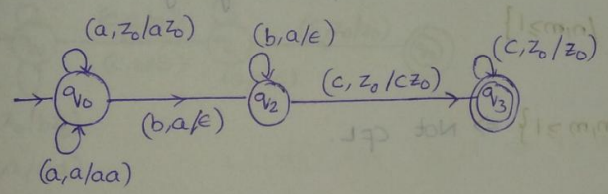
② $w \in (arb) / n_a(w) = n_b(w)$

Let the string be abbbbaa
 $\uparrow \uparrow \uparrow \uparrow \uparrow$



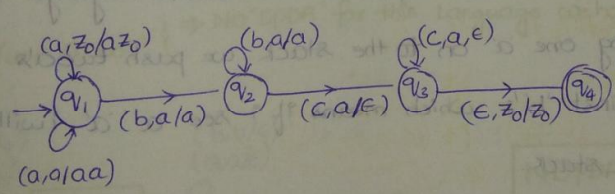
- 1> push a's onto the stack.
- 2> If you see b pop one 'a' & vice versa.
- 3> If your Alphabet is 'b' & Bottom of Stack = z0
 $\Rightarrow$ push 'b' on stack.

③ $L = \{a^n b^n c^m / n, m \geq 1\}$

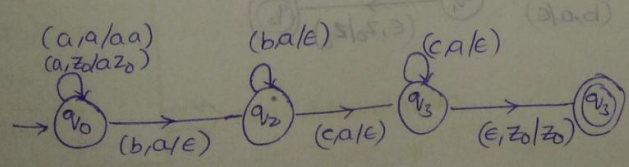


L-69

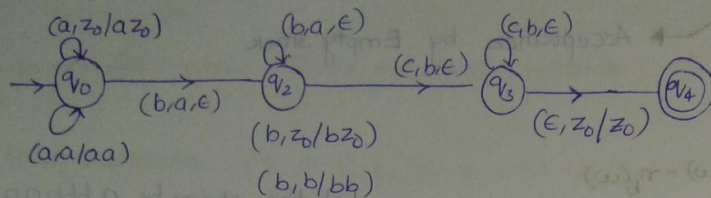
④ $L = \{a^n b^m c^n / n, m \geq 1\}$



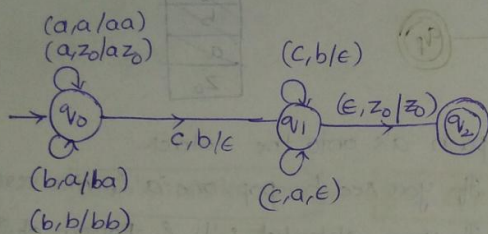
⑤ $L = \{a^{m+n} b^m c^n\} (m, n \geq 1)$



$$⑥ L = \{a^n b^{m+n} c^m / n, m \geq 1\}$$



$$⑦ L = \{a^n b^m c^{n+m} / n, m \geq 1\}$$



$$⑧ L = \{a^n b^n c^m d^m / n, m \geq 1\}$$

⇒ context free languages

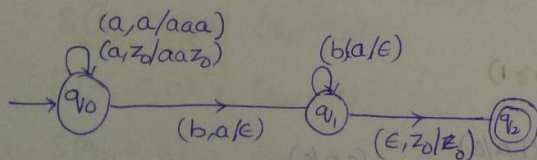
$$L = \{a^n b^m c^m d^n / n, m \geq 1\}$$

$$⑨ L = \{a^n b^m c^n d^m / n, m \geq 1\} \Rightarrow \text{Not CFL.}$$

$$⑩ L = \{a^n b^{2n} / n \geq 1\}$$

The language accepted is $L = \{abb, aabbbb, aaabbbbbbb, \dots\}$

⇒ The solution to construct PDA for the above language is instead of pushing one 'a' on to the stack we push two 'a's and pop them against 'b's which means if 'i' see 'a' 'a' i will push 'z's onto the stack.



⇒ The other should pushing 2 b's

$$⑪ L = \{a^n b^n\}$$

$$⑫ L = \{a^n b^n\}$$

$$L = \{a^n b^n\}$$

$$L = \{a^n b^n\}$$

$$(b, z_0/bz_0)$$



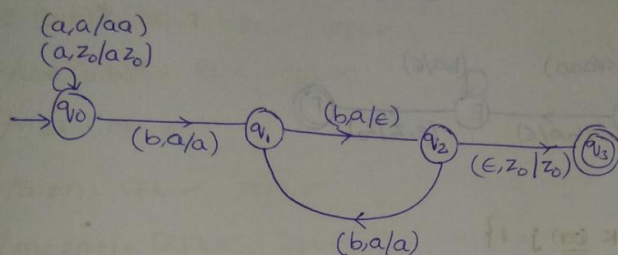
$$(a, b/ab)$$

$$(b, a/ab)$$

$$(a, a/aa)$$

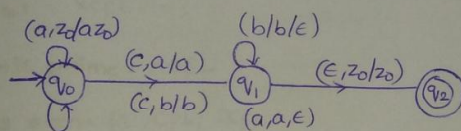
$$(b, b/bb)$$

⇒ The other solution for the above problem is for every two b's we should pop 1 a. which means we push all the a's and for pushing 'b' on to the stack we pop 1a against the occurrence of 2 b's



① $L = \{a^n b^n c^n / n \geq 1\} \Rightarrow \text{Not Rego CFL}$

② $L = \{w c w^R / w \in (a,b)^+\}$



(b,z₀/b z₀)

(a,a/aa)

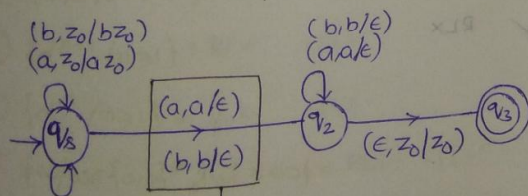
(a,b/ab)

(b,a/ba)

(b,b/bb)

L=70

$L = \{w w^R / w \in (a+b)^+\} \Rightarrow \text{No "DPDA" for this language we have to construct "NDPDA"}$



(a,b/ab)

(b,a/ba)

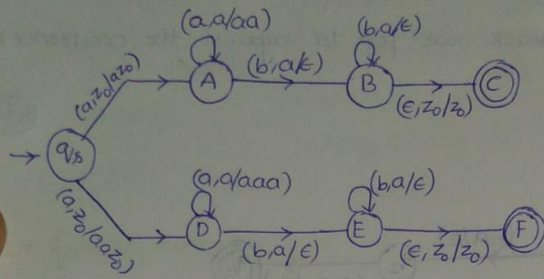
(a,a/aa)

(b,b/bb)

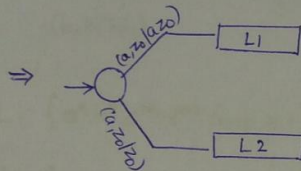
Non-determinism.

NDPDA is not as powerful
as DPDA

$$L = \{a^n b^n / n \geq 1\} \cup \{a^n b^{2n} / n \geq 1\}$$



$$\begin{aligned} \textcircled{3} L &= \{a^i b^j c^k d^l / i=k \text{ or } j=l\} \\ &= \{a^m b^j c^m d^l\} \cup \{a^i b^m c^k d^m\} \end{aligned}$$



L-71

$$\textcircled{1} a^{m+n} b^n c^m / n, m \geq 1 \Rightarrow \text{RL} \times \text{CFL} \checkmark (\text{DCFL})$$

$$a^m b^{m+n} c^n / n, m \geq 1$$

$$\textcircled{2} a^m b^n c^{m+n} = \text{RL} \times \text{CFL} \checkmark$$

$$\textcircled{3} a^m b^m c^n d^n = \text{RL} \times \text{CFL} \checkmark$$

$$\textcircled{4} a^m b^n c^m d^n / m, n \geq 1 = \text{RL} \times \text{CFL} \times$$

$$\textcircled{5} a^m b^n c^n d^m / m, n \geq 1 = \text{CFL} \checkmark \text{RL} \times$$

$$\textcircled{6} a^m b^i c^m d^k / m, n \geq 1 = \text{DCFL} \checkmark \text{RL} \times$$

$$\textcircled{7} a^m b^n / m > n = \text{DCFL} \checkmark$$

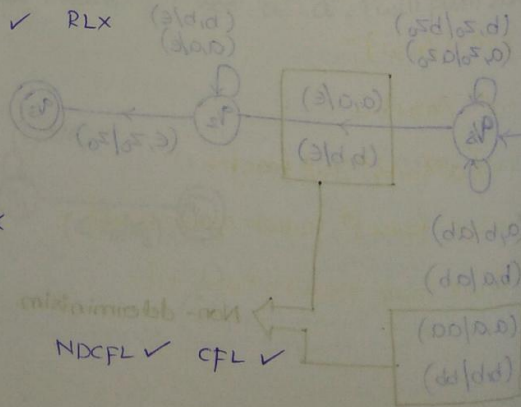
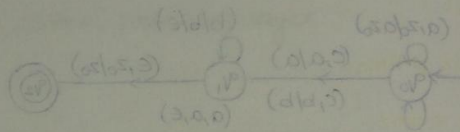
$$\textcircled{8} a^n b^{2n} / n \geq 1 = \text{DCFL} \checkmark$$

$$\textcircled{9} a^n b^{n^2} / n \geq 1 = \text{RL} \times \text{CFL} \times$$

$$\textcircled{10} a^n b^{2^n} / n \geq 1 = \text{RL} \times \text{CFL} \times$$

$$\textcircled{11} ww^R / w \in (a,b)^+ = \text{RL} \times \text{DCFL} \checkmark \text{NDCFL} \checkmark \text{CFL} \checkmark$$

$$\textcircled{12} ww / w \in (a,b)^+ = \text{RL} \times \text{CFL} \times$$



$$\textcircled{13} a^n b^n c^m / m$$

$$\textcircled{14} a^n b^n c^n d^n$$

$$\textcircled{15} a^n b^{2n} c^{3n} / n$$

$$\textcircled{16} xcy / x, y \in \Sigma^*$$

$$\textcircled{17} xx^R / x \in \Sigma^*$$

$$\textcircled{18} ww^R / w \in \Sigma^*$$

$$\textcircled{19} a^n b^{3^n} / n \geq 1$$

$$\textcircled{20} a^m b^n / m \neq n$$

$$\textcircled{21} a^m b^n / m \neq n$$

$$\textcircled{22} a^i b^j / i \neq j$$

$$\textcircled{23} a^{n^2} / n \geq 1$$

$$\textcircled{24} a^{2^n} / n \geq 1$$

$$\textcircled{25} a^n / n \geq 1$$

$$\textcircled{26} a^m / m \text{ is even}$$

$$\textcircled{27} a^i b^j c^k / i \geq j$$

$$\textcircled{28} a^i b^j c^k / i \geq j$$

$$\textcircled{29} a^i b^j c^k d^l / i \geq j$$

$$\textcircled{30} a^i b^j c^k d^l / i \geq j$$

$$\textcircled{31} a^i b^j c^k d^l / i \geq j$$

$$\textcircled{32} a^m b^i c^k d^n$$

$$\textcircled{33} a^n b^4 m / n$$

$$\textcircled{34} \{a^3, a^8, a^{13}, \dots\}$$

$$\textcircled{35} \{a^{2n+1} / n \geq 1\}$$

$$\textcircled{36} \{a^n / n \geq 1\}$$

$$\textcircled{37} \{w / w \in \Sigma^*\}$$

$$\textcircled{38} \{w / w \in \Sigma^*\}$$

$$\textcircled{39} \{w / w \in \Sigma^*\}$$

$$\textcircled{40} \{w / w \in \Sigma^*\}$$

(34)

$$a^n b^n c^m / n > m \quad \text{XRL} \quad \text{XCFL} \quad \text{XCFLL}$$

$$a^n b^n c^n d^n / n \leq 10^{10} = \text{RLV} \quad \text{DCFLV} \quad \text{CFLV}$$

$$a^n b^{2n} c^{3n} / n \geq 1 = \text{RLX} \quad \text{CFLX}$$

$$xcy / x, y \in (0,1)^+ = \text{RLV} \quad \text{DCFLV} \quad \text{CFLV}$$

$$xx^R / x \in (a,b)^*, |x| \leq 1 = \text{RLV} \quad \text{CFLV}$$

$$www^R / w \in (a,b)^* = \text{RLX} \quad \text{CFLX}$$

$$a^n b^{3n} / n \geq 1 = \text{CFLX}$$

$$a^m b^n / m \neq n = \text{CFL} \checkmark \quad \text{DCFLV}$$

$$a^m b^n / m = 2n+1 = \text{DCFLV} \quad \text{CFLV}$$

$$a^i b^j / i \neq 2j+1 = \text{DCFLV}$$

$$a^{n^2} / n \geq 1 = \text{CFLX}$$

$$a^{2^n} / n \geq 1 = \text{XCFL}$$

$$a^n b / n \geq 1 = \text{XCFL}$$

$$a^m / m \text{ is prime} = \text{XCFL}$$

$$a^k / k \text{ is even} = \text{RL, CFL, DCFL}$$

$$a^i b^j c^k / i \geq j \geq k = \text{XCFL}$$

$$a^i b^j c^k / j = i+k = \text{CFL, DCFL} \checkmark$$

$$a^i b^j c^k d^l / i=k \text{ or } j=1 = \text{CFLV}$$

$$a^i b^j c^k d^l / i=k \text{ and } j=1 = \text{CFLX}$$

$$a^m b^l c^k d^n / m, l, k, n \geq 1 = \text{RLV} \quad \text{CFLV}$$

$$a^n b^4 m / n, m \geq 1 = \text{RLV} \quad \text{CFLV}$$

$$\{a^3, a^8, a^{13}, \dots\} = \text{RLV} \quad \text{CFLV}$$

$$\{a^{2n+1} / n \geq 1\} = \text{RLV} \quad \text{CFLV}$$

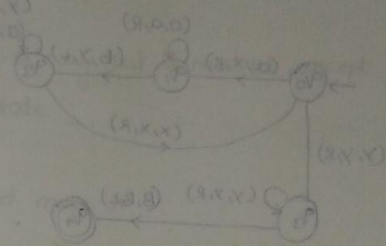
$$\{a^n / n \geq 1\} = \text{RLX} \quad \text{CFLX}$$

$$\{w / w \in \{a,b\}^*, |w| \geq 100\} = \text{RLV} \quad \text{CFLV}$$

$$\{w / w \in \{a,b,c\}^*, \{n_a(w) = n_b(w) = n_c(w)\}\} = \{a^n b^n c^n / n \geq 1\} \times \text{CFL}$$

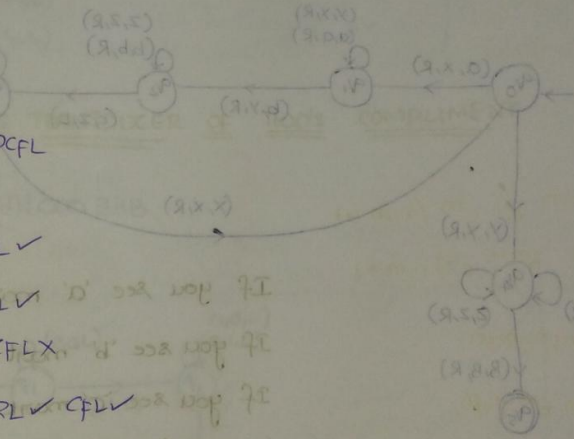
$$\{w / w \in \{a,b\}^*, \{n_a(w) \geq n_b(w) + 1\}\} = \text{CFLV}$$

(35)



TRANSUCER CONVERTED

(1,2,3,4,5,6,7,8,9,10) = 1



(a,x,1,p) = (a,0,p)8

(a,x,0,p) = (a,0,p)8

(56)

3. TURING MACHINE AS TRANSDUCER OF ONES COMPLIMENT

(57)

⇒ TM has the capacity to Read from the tape and also write on the tape

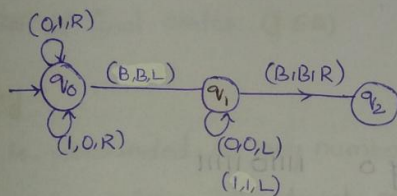
⇒ TM to find ones complement of a Binary number

⇒ Here the TM is designed in a such a way that it need not accept anything so there will not be final state

⇒ If i see a '0' mark it as '1' and move 'R'

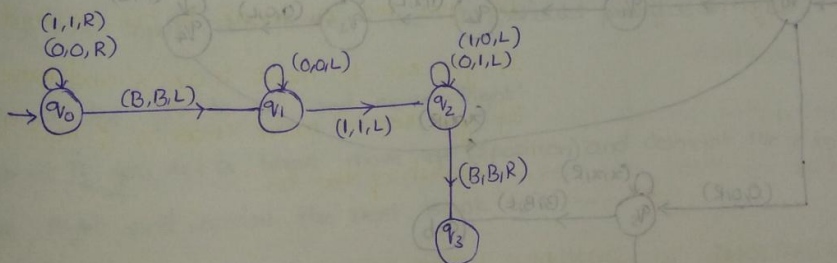
⇒ If i see a '1' mark it as '0' and move 'R'

TRANSDUCER = CONVERTER



4. TURING MACHINE AS TRANSDUCER OF TWO'S COMPLIMENT

The Given string BBB 0111000 BBB



⇒ start from the starting point of Input symbol and move 'R' till the end

⇒ Now start moving Left without changing anything until you see the

1st '1' (one)

⇒ Now, the Remaining bits to the Left of bits should be complemented.

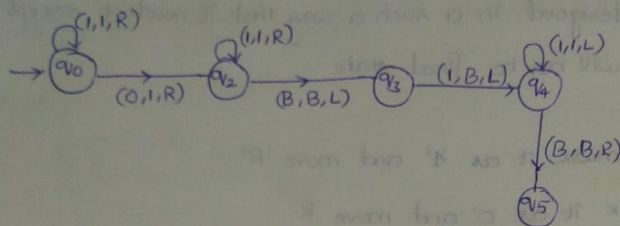
TM AS AN ADDER (OF UNARY NUMBERS)

$$(3) = (111)_1 \Rightarrow |x| = 3$$

and both x, y are separated by '0'

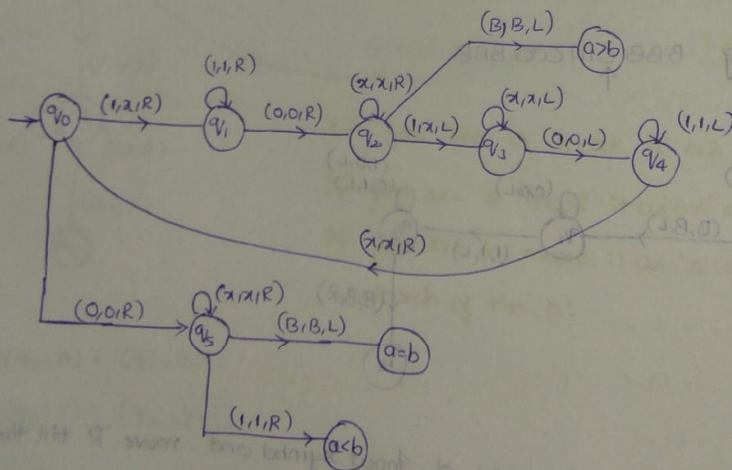
$$(4) = (1111)_1 \Rightarrow |y| = 4$$

$$(11101111)$$



TM AS A COMPARATOR

$$\left. \begin{array}{l} a = 4 = (1111)_1 \\ b = 5 = (11111)_1 \end{array} \right\} a, b \text{ are separated by } 0 \quad 111101111$$



TM can perform any mathematical function

THE STANDARD

$$M = (Q, \Sigma, \Gamma)$$

Q = Set of states

Σ = Input alphabet

Γ = Tape Alphabet

δ = Transition function

q_0 = Initial state

B = is used to

F = set of final

Summary

1) Tape is unbounded

2) It is deterministic

3) No special input

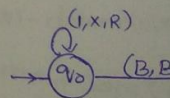
TM AS A COPIER

copier means that it copies the input. If the input is 111101111

→ Make all the

→ And now if you

→ Move Right



THE STANDARD TURING MACHINE

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Q = Set of states

Σ = Input alphabet

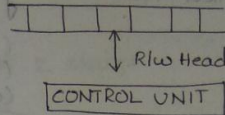
Γ = Tape alphabet

δ = Transition function

q_0 = Initial state ($q_0 \in Q$)

B is used to Represent Blank ($B \in \Gamma$)

F = Set of final states. ($F \subseteq Q$)



$$\delta: (Q \times \Gamma) \rightarrow (Q \times \Gamma \times \{L, R\})$$

Summary

- 1) Tape is unbounded, so any number of left and Right moves are possible
- 2) It is deterministic, i.e. atmost one move for each configuration.
- 3) NO special i/p or o/p file.

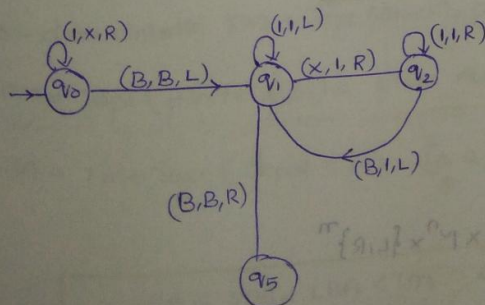
8) TM AS A COPIER

copier means the Turing machine should produce the copy of the given input. If the input is $B11B$ then the TM should produce $B1111B$

→ Make all the 1's to 'x' and move 'Right'

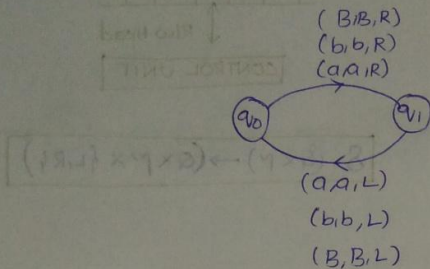
→ And now if you see a blank move left (1 position) and convert the x to 1'

→ Move Right and convert the next Blank to 1'. ---



9) NON-HALTING TURING MACHINE

There are some Turing machines that will never halt for some inputs they are called Non-Halting Turing machines.



10)

TURING THESIS

Any computation that can be carried out by mechanical means can be performed by some Turing machine.

⇒ Anything that can be done by existing digital computers can also be done by TM.

⇒ No one has yet been able to suggest a problem, solvable by what we intuitively considering an Algorithm, for which TM program cannot be written.

⇒ Alternative models have been proposed for mechanical computation, but none of them are more powerful than the Turing machine model.

11)

MODIFICATIONS TO STANDARD TURING MACHINE

1) TM with stay option

2) TM with semi infinite tape

3) Offline TM

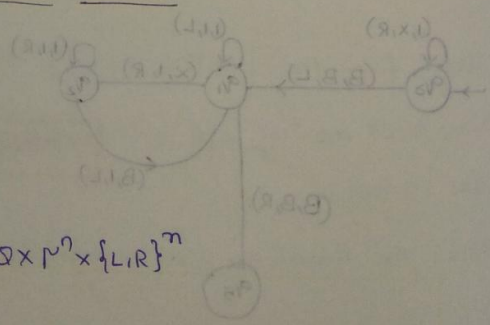
4) Multitape TM

$$\delta: Q \times \Sigma^n \rightarrow Q \times \Sigma^n \times \{L, R\}^n$$

5) Jumping TM

6) Non-Erasing TM

7) Always writing TM



8) Multidimensional

9) Multitape TM

10) Automata with

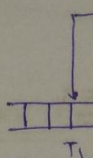
11) TM with only

12) Multitape TM

13) Non-deterministic

14) A NPDA with

15) UNIVERSAL TM



(Rep of TM)

$$\delta(q_1, 1) =$$

$$\delta(q_2, 1) =$$

→ Every "TM"

13) LINEAR BOUN

⇒ Non-deterministic

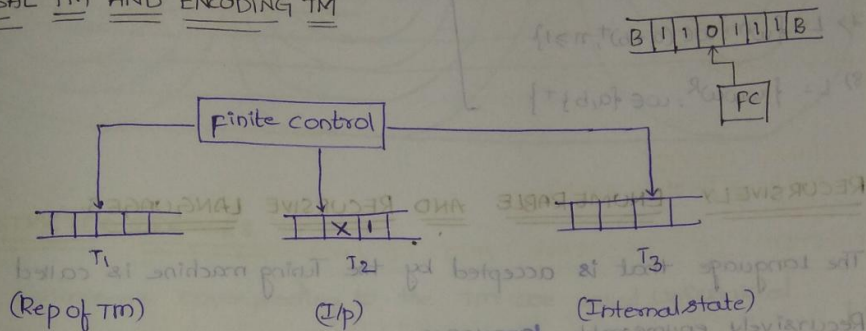
⇒ NTM + Tape

⇒ LBA = TM +

60

- 5) Multidimensional TM $S: Q \times \Sigma^n \rightarrow Q \times \Sigma^n \times \{L, R, U, D\}$
- 6) Multitape TM
- 7) Automata with a Queue
- 8) TM with only 3 states
- 9) Multitape TM with stay option and atmost 2 states
- 10) Non-deterministic TM $S: Q \times \Sigma \rightarrow Q \times \Sigma \times \{L, R\}$
- 11) A NPDA with 2 independent stacks $S: Q \times (\Sigma \cup \epsilon)^* \times \Sigma^* \times \Sigma^* \rightarrow Q \times \Sigma^* \times \Sigma^*$

12) UNIVERSAL TM AND ENCODING TM



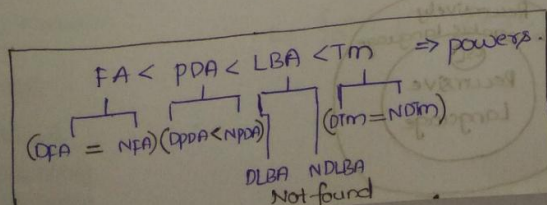
$$\begin{aligned}
 S(q_1, 1) &= (q_2, X, R) & Q &= (q_1, q_2, q_3, q_4, \dots) & q_1 &= 1 & a_1 &= 1 \\
 S(q_2, 1) &= (q_3, Y, R) & R &= (r_1, r_2, r_3, r_4, \dots) & q_2 &= 11 & a_2 &= 11 \\
 & & & & q_3 &= 111 & a_3 &= 111 \\
 & & & & q_4 &= 1111 & a_4 &= 1111 \\
 & & & & R &= 11 & L &= 1
 \end{aligned}$$

Below the equations, a sequence of states is shown: $S(q_1, a_1) = (q_2, a_2, R)$ with a sequence of states: 01010110110110 .

→ Every "TM" can be represented as the string of 0's and 1's.

13) LINEAR BOUNDED AUTOMATA

- ⇒ Non-deterministic TM + Tape (stack) = PDA
- ⇒ NTM + Tape (finite size) = Finite Automata
- ⇒ LBA = TM + Tape (Input size) $[a \ a \ b \ b]$



The some of the Languages that are accepted by the LBA are

62

$$1) L = \{a^n b^n c^n : n \geq 1\}$$

$$2) L = \{a^n : n \geq 0\}$$

$$3) L = \{a^n : n = m^2, m \geq 1\}$$

$$4) L = \{a^n : n \text{ is prime}\}$$

$$5) L = \{a^n : n \text{ is not prime}\}$$

$$6) L = \{ww : w \in \{a,b\}^+\}$$

$$7) L = \{w^n : w \in \{a,b\}^+, n \geq 1\}$$

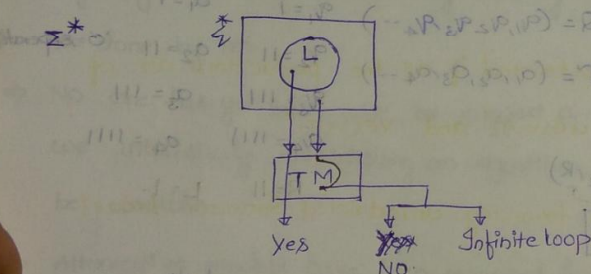
$$8) L = \{www^R : w \in \{a,b\}^+\}$$

Context Sensitive Languages

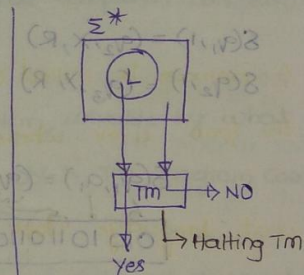
"LBA" is a "Halting TM"

RECURSIVELY ENUMERABLE AND RECURSIVE LANGUAGES

The language that is accepted by the Turing machine is called Recursively enumerable languages



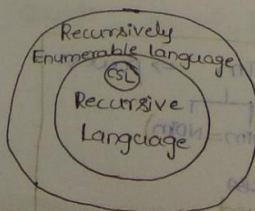
Recursively Enumerable Languages



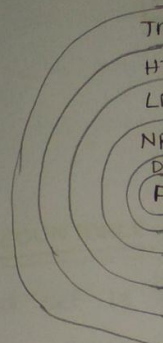
Recursive Language

⇒ If there exist a Turing machine for a language then the language is called Recursively Enumerable language.

⇒ If there exists a Halting TM for a language then the language is called Recursive language



THE BIG PICTURE



UNRESTRICTED

⇒ The Grammar

Grammars

⇒ A Grammar

the form

① what language

$$S \rightarrow S_1 B$$

$$S_1 \rightarrow a S, b$$

$$b B \rightarrow b b b B$$

$$a S, b \rightarrow a a$$

$$B \rightarrow \lambda$$

so: $S \Rightarrow S_1$

$$\Rightarrow a S$$

$$\Rightarrow a a$$

$$\Rightarrow a a$$

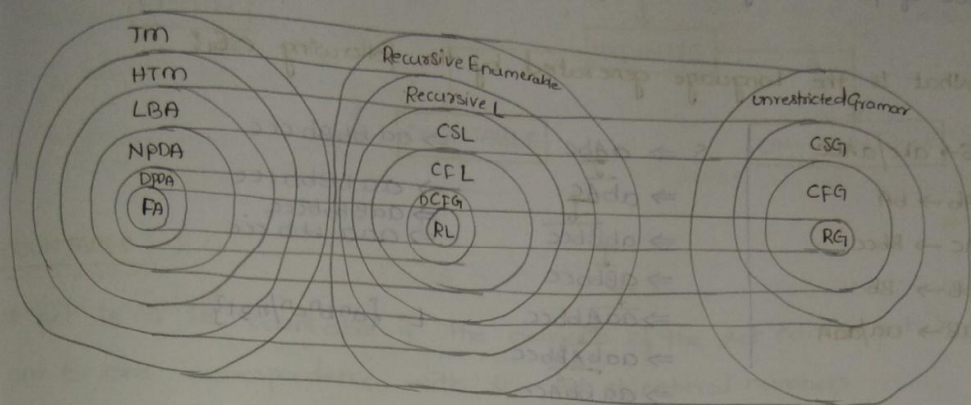
$$S \Rightarrow S_1 B$$

$$\Rightarrow a S, b B$$

$$\Rightarrow a S, b b b B$$

$$\Rightarrow a a b b b B$$

$$\Rightarrow a a b b b$$



UNRESTRICTED GRAMMARS

⇒ The Grammars corresponding to the TM are called Unrestricted Grammars.

$\Rightarrow$ A Grammar is called unrestricted if all the productions are of the form $u \rightarrow v$ where $u \in (V \cup T)^+$ and $v \in (V \cup T)^*$

① what language does the following unrestricted Grammar derive?

$$S \rightarrow S_1 B$$
$$S_1 \rightarrow aS_1, b$$
$$bB \rightarrow bbbB$$
$$as, b \rightarrow aa$$
$$B \rightarrow \lambda$$
$$\underline{\underline{S_0}}: S \Rightarrow S_1 B$$
$$\Rightarrow a \in A, b \in B$$
$$\Rightarrow aa \ B$$
$$\Rightarrow aa$$
$$S \Rightarrow S_1 B$$
$$\Rightarrow aS, bB$$
$$\Rightarrow a s_1 b b b B$$
$$\Rightarrow aa bbb B$$

$$L = \{a^{n+1} b^{n+k} / n \geq 1, k = -1, 1, 3, 5, \dots\}$$

CONTEXT SENSITIVE GRAMMAR - EXAMPLE 1

64

A Grammar is said to be Context sensitive if all the productions are of form $x \rightarrow y$ where $(x, y) \in (V \cup T)^+$ and $|x| \leq |y|$

What is the language generated by the following CSG?

$S \rightarrow abc/aAbc$	$S \Rightarrow aAbc$	$\Rightarrow aa b b b c c c$
$Ab \rightarrow bA$	$\Rightarrow abAc$	$\Rightarrow aa b b b b c c c$
$Ac \rightarrow Bbcc$	$\Rightarrow abBbcc$	$\Rightarrow aa b b b b c c c$
$bB \rightarrow Bb$	$\Rightarrow a b B b c c$	$\Rightarrow a a a b b b c c c$
$aB \rightarrow aa/a a A$	$\Rightarrow aa A b b c c$	
	$\Rightarrow a a b A b b c c$	$L = \{a^n b^n c^n / n \geq 1\}$
	$\Rightarrow a a b b A c c$	

EXAMPLE-2 [Not in GATE]

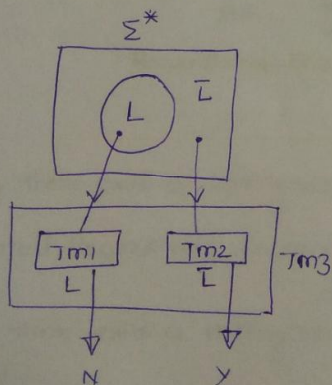
THEOREM ON RECURSIVE AND RE LANGUAGES

Recursively Enumerable

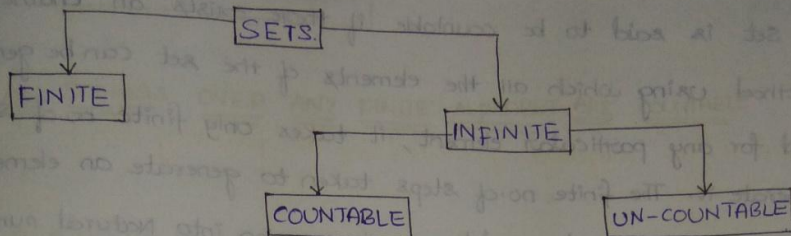
If a language 'L' and its Complement $\bar{L}$ are both RE, then both the languages are Recursive. If 'L' is recursive, then $\bar{L}$ is also recursive and consequently both are Recursively enumerable.

$\Rightarrow$ There is no membership Algorithm for RE Languages.

$\Rightarrow$ Membership Algorithm exists for the Recursive Languages



COUNTABILITY



COUNTABLE

A set is 'S' is countable if the elements of the set can be put in one to one correspondence with the set of natural numbers

UNCOUNTABLE

A set is uncountable if it is infinite and not countable.

$$E = \{0, 2, 4, 6, 8, 10, 12, \dots\}$$

$$N = \{1, 2, 3, 4, 5, 6, \dots\}$$

$$\begin{matrix} 2i \\ \downarrow \\ i+1 \end{matrix}$$

One to one correspondence is there
so countable

$$O = \{1, 3, 5, 7, 9, 11, \dots\}$$

$$N = \{1, 2, 3, 4, 5, 6, \dots\}$$

$$\begin{matrix} 2i+1 \\ \downarrow \\ i+1 \end{matrix}$$

One-one correspondence is there so
countable

Real No's $\{0 \text{ --- } 1\}$

→ Infinite No's are present and could eatup all the Natural

No's set so set of Real numbers is uncountable

→ There are some languages for which Turing machines cannot be constructed. ⇒ There are some languages which are not Recursively

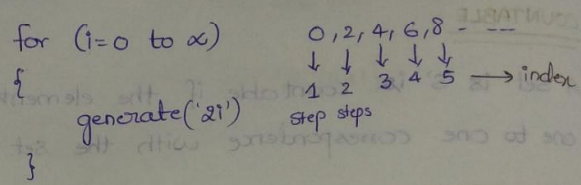
Enumerable

ALTERNATIVE DEFINITION OF COUNTABILITY

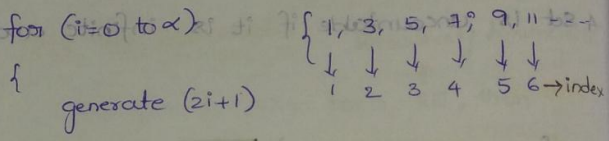
66

A set is said to be countable if there exists an enumeration method using which all the elements of the set can be generated and for any particular element, it takes only finite no. of steps to generate it. The finite no. of steps taken to generate an element can be used as its index and hence a mapping into Natural numbers.

Set of all Even numbers = for $(i=0 \text{ to } \infty)$



Set of all odd numbers = for $(i=0 \text{ to } \infty)$



Set of all Real numbers = No Enumeration method so not countable

SET OF ALL QUOTIENTS ARE COUNTABLE

Quotients of P/Q where $(P, Q) \in \mathbb{Z}^+$

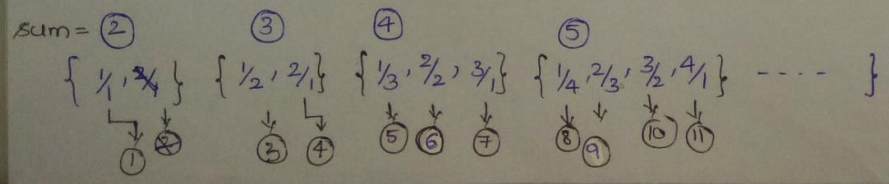
$S = \{1/1, 1/2, 2/1, 1/3, 1/4, 3/4, \dots\}$

$= \{1/1, 1/2, 1/3, 1/4, 1/5, 1/6, \dots\} \times 2/1$ will not be generated so this

Enumeration method is wrong.

so follow this enumeration method, of (P/Q)

1) find the sum of $(P+Q)$ and go in increasing order of the sum



So Every P_q can be generated after finite no. of steps and therefore the

set of all quotients are countable

(67)

SET OF ALL STRINGS OVER ANY FINITE ALPHABET ARE COUNTABLE

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$$

Now the Enumeration method that must be followed is

1) first Generate all the strings of length 0 (' ϵ ') and now generate all the strings of length 1 $\{a, b\}$ and in the set of length $\{1\}$ follow the Lexicographic order.

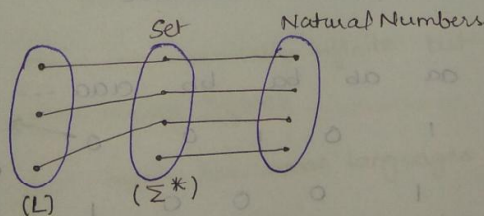
$$S = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

$$\text{Steps} = \begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow \\ \{1 & 2 & 3 & 4 & \dots\} \end{matrix}$$

$\therefore$ SET OF ALL POSSIBLE STRINGS OVER ANY FINITE ALPHABET SET ARE COUNTABLE, i.e. Σ^* IS COUNTABLE

Now, Another important property is.

Every subset of countable set is either finite or countable



Now we know that Σ^* is countable and $L \subseteq \Sigma^*$ therefore

"Every language is countable" $\rightarrow$ NOT ALL LANGUAGES

$\rightarrow$ GIVEN A LANGUAGE ITS STRING CAN BE PUT IN ONE-ONE CORRESPONDANCE OF NATURAL NO'S.

SET OF ALL TURING MACHINES ARE COUNTABLE

$\Sigma = \{0,1\}$ (1) $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$ = countable

(2) Every TM can be encoded as a strings of 0s and 1s

(3) Set of all TM's (S) $\{S \in \Sigma^*\}$

$\Leftrightarrow$ Every subset of countable set is finite or countable

$\therefore$ SET OF ALL TM'S ARE COUNTABLE

IMPLICATIONS OF THE FACT THAT THE TURING MACHINES ARE COUNTABLE

TM's are countable

$\Rightarrow$ Recursively Enumerable Languages are Countable

$\Rightarrow$ Recursive Languages are countable

$\Rightarrow$ CSL are countable $\Rightarrow$ LBA are Countable

$\Rightarrow$ CFL are countable $\Rightarrow$ PDA are Countable

$\Rightarrow$ RL are countable $\Rightarrow$ Finite Automata are countable

DIAGONALIZATION METHOD TO PROVE SET OF ALL LANGUAGES ARE

UNCOUNTABLE

$\Sigma = \{a,b\}$ Σ^* is countable, 2^{Σ^*} is uncountable

Σ^*	ϵ	a	b	aa	ab	ba	bb	aaa	...
1)	0	1	1	1	0	1	0	0	...
2)	0	1	0	1	0	0	0	1	...
3)	1	0	1	1	1	0	1	0	...
4)	1	0	0	0	0	1	0	1	...
5)	0	1	1	0	0	1	1	0	...
6)	1	0	1	0	1	0	1	0	...
7)	1	1	1	1	0	0	0	0	...
8)	0	1	1	1	0	1	1	1	...

Represent ele in power set $\therefore \{a,b\}^*$

Complete set $\{0,1\}^*$

Not in the language

Set of Languages possible are uncountable

All TM's are countable but all the Languages possible are uncountable

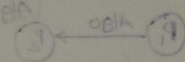
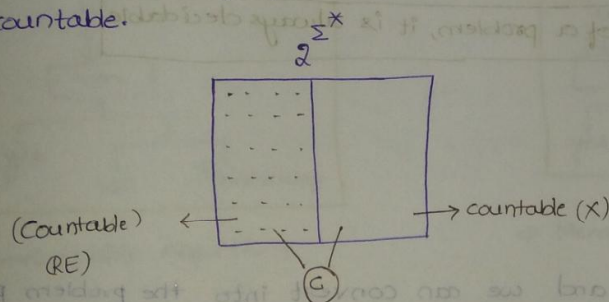
There exist some languages which a Turing machine cannot Accept.

SOME PROBLEMS ON COUNTABILITY

⇒ If S_1 and S_2 are countable sets, then $S_1 \cup S_2$ is countable and $S_1 \times S_2$ is countable

⇒ The cartesian product of finite no of countable sets is countable

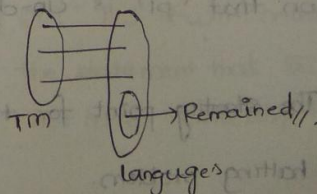
⇒ The set of all languages that are not recursively Enumerable is uncountable.



→ If S is countably infinite then 2^S is uncountable
 put $S = \Sigma^*$ ⇒ If Σ^* is countably infinite then 2^{Σ^*} (set of all languages) are uncountable (uncountable infinite)
 power set {set of all subsets possible}

Now, TM - are countably infinite but all languages are uncountably infinite

∴ There are some languages for which TM does not exist.



in power
library

0 | Comp
0 | ment
the language

COMPUTABILITY AND DECIDABILITY

70

Computability

and

Decidability

function

$$f(n) = (n^2 + 1)$$

Algo (or) TM (halts) ✓

Tape

Problem

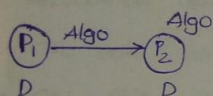
A stmt whose True or false then it is decidable, ans is either

Ex: $\exists n$ is a prime

D = set of all Natural no's

Ex: A given Grammar G is Ambiguous $\Rightarrow$ undecidable
Domain = {Set of all CFG's}

Given an instance of a problem, it is always decidable



Given an problem 'p1' and we can convert into the problem p2 using an Algorithm and the problem 'p2' can be solved using another Algorithm, which means that there exists an Algorithm for problem 'p1'. The conversion should be in such a way that the solution to 'p2' should also be a solution to p1. which means

if the ^{sol to} problem p2 is True, then the solution to p1 should also be True.

And the procedure of conversion is called "Reducability". If you find out that if 'p2' is decidable then 'p1' is also decidable. But before it is known that 'p1' is undecidable then the problem 'p2' is undecidable

$\Rightarrow$ The starting point for the problem of decidability is the halting problem.

TM - HALTING

Given the d
started with
halts.

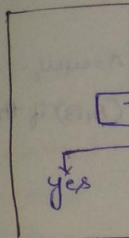
Theorem

If the halting

be Recursive

Recursively

TM



$\Rightarrow$ No member

consider a

Σ^*

$L = RE$

$TM = m$

(M, w)

TM- HALTING PROBLEM

Given the description of a TM 'M' and an input 'w' does 'M' when started with 'w', does 'M' when started with 'w' as its input eventually halts.

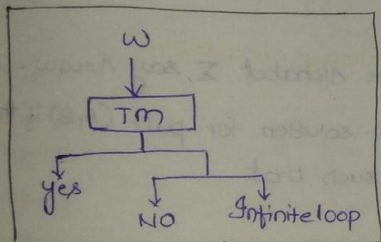
Theorem

If the halting problem were decidable, then every RE language would be Recursive. Consequently, the halting problem is undecidable.

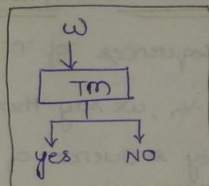
Recursively Enumerable

Recursive language

TM



TM which halts.

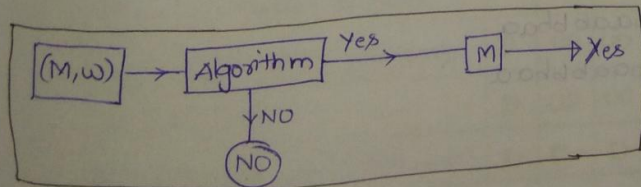
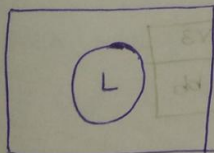


⇒ No membership Algorithm

⇒ Membership Algo exists

Consider a Language 'L' which is Recursively Enumerable
(No halting TM exists)

Σ^*
L = RE
TM = M



∴ This is the membership Algorithm we got for the "REL" but it will contradict the statement that "REL do not have Membership Algo".

SOME UNDECIDABLE PROBLEMS BASED ON TM HALTING PROBLEM

1) The state entry problem is, given a TM, a state $q \in Q$ and $w \in \Sigma^+$, decide whether or not the state q is ever entered when M is applied to w . This is undecidable.

2) Given a TM M , whether or not M halts if started with a blank space. This is undecidable.

3) Almost any problem related to RE languages is undecidable.

POST CORRESPONDENCE PROBLEM

Given two sequences of n strings on some Alphabet Σ , say $A = w_1 w_2 \dots w_n$ and $B = v_1 v_2 \dots v_n$, we say there exists a pc-solution for pair (A, B) if there is a non empty sequence of integers i, j, k such that

$$w_i w_j \dots w_k = v_i v_j \dots v_k$$

pc-problem is to devise an algorithm that will tell us for any (A, B) , whether or not there exist a pc-solution.

Ex:

$$A = \begin{array}{|c|c|c|} \hline w_1 & w_2 & w_3 \\ \hline a & ab & bba \\ \hline \end{array}$$

$$B = \begin{array}{|c|c|c|} \hline v_1 & v_2 & v_3 \\ \hline baa & aa & bb \\ \hline \end{array}$$

The post correspondence solution is $(3, 2, 3, 1)$

$$w_3 w_2 w_3 w_1 = bbaabbbbaa$$

$$v_3 v_2 v_3 v_1 = bbaabbbbaa$$

COMPLEXITY

P-CLASS

The set of TM in poly

NP-CLASS

The set of time is cal

$$(a \vee b \vee c)$$

$$2 \times 2 \times 2$$

$$= (2^n)$$

$$= O(2^n)$$

DECIDABILITY

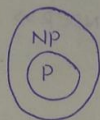
COMPLEXITY CLASSES

P-CLASS

The set of all languages that are accepted by some deterministic TM in polynomial time

NP-CLASS

The set of all languages accepted by Non-deterministic TM in polynomial time is called Np-class.



NDTM $\rightarrow$ DTM

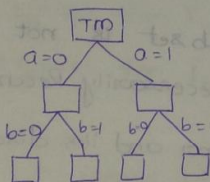
$O(n^k) \rightarrow O(2^n)$

(avbvc)

$2 \times 2 \times 2$

$= 2^n$

$= O(2^n)$



DECIDABILITY TABLE

PROBLEM	RL	DCL	CFL	CSL	Recurs ive lan guage	REL
Does w ∈ L? (Membership Algo)	D	D	D	D	D	UD
Is L = ∅? (Emptiness problem)	D	D	D	UD	UD	UD
L = Σ* (Completeness problem)	D	UD	UD	UD	UD	UD
Is L ₁ = L ₂ (Equality problem)	D	UD	UD	UD	UD	UD
Is L ₁ ⊆ L ₂ (Subset problem)	D	UD	UD	UD	UD	UD
Is L ₁ ∩ L ₂ = ∅	D	UD	UD	UD	UD	UD
Is L finite or not? (Finiteness)	D	D	D	UD	UD	UD
Is complement of L a language of same type or not?	D	D	D	UD	UD	UD
Is intersection of 2 languages of same type	D	UD	UD	UD	UD	UD
Is L Regular Language	D	D	UD	UD	UD	UD

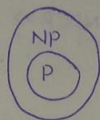
COMPLEXITY CLASSES

P-CLASS

The set of all languages that are accepted by some deterministic TM in polynomial time

NP-CLASS

The set of all languages accepted by Non-deterministic TM in polynomial time is called Np-class.



NDTM $\rightarrow$ DTM

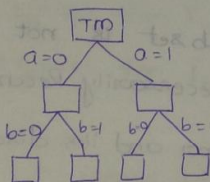
$O(n^k) \rightarrow O(2^n)$

(avbvc)

$2 \times 2 \times 2$

$= 2^n$

$= O(2^n)$



DECIDABILITY TABLE

PROBLEM	RL	DFL	CF	CSL	Recurs ive lan guage	REL
Does w ∈ L? (Membership Algo)	D	D	D	D	D	UD
Is L = ∅? (Emptiness problem)	D	D	D	UD	UD	UD
L = Σ* (Completeness problem)	D	UD	UD	UD	UD	UD
Is L ₁ = L ₂ (Equality problem)	D	UD	UD	UD	UD	UD
Is L ₁ ⊆ L ₂ (Subset problem)	D	UD	UD	UD	UD	UD
Is L ₁ ∩ L ₂ = ∅	D	UD	UD	UD	UD	UD
Is L finite or not? (Finiteness)	D	D	D	UD	UD	UD
Is complement of L a language of same type or not?	D	D	D	UD	UD	UD
Is intersection of 2 languages of same type	D	UD	UD	UD	UD	UD
Is L Regular Language	D	D	UD	UD	UD	UD

74

CFL $\Rightarrow$ closed under union

Concatenation

Closure

75

$$\begin{matrix} L_1 & L_2 \\ S_1 \rightarrow & S_2 \rightarrow \end{matrix}$$

$$\text{CFL} \left\{ \begin{matrix} S \rightarrow S_1/S_2 \\ S_1 \rightarrow \\ S_2 \rightarrow \end{matrix} \right.$$

$$\begin{matrix} L_1 & L_2 \\ S_1 \rightarrow & S_2 \rightarrow \\ S \rightarrow S_1 \cdot S_2 \\ S_1 \rightarrow \text{CFL} \\ S_2 \rightarrow \end{matrix}$$

$$\begin{matrix} L_1 & L_1^* \\ S_1 \rightarrow & \downarrow \\ & S \rightarrow S_1 S_1 \in \text{CFL} \end{matrix}$$

Not closed under intersection $\Rightarrow L_1 \quad L_2$

$$L_1 = \{a^n b^n c^m / m, n \geq 0\}$$

$$L_2 = \{a^n b^n c^n / n \geq 0\}$$

$$L_2 = \{a^n b^n c^n / n, m \geq 0\}$$

NOT CFL

Not closed under complementation $= \overline{L_1 \cap L_2} = \overline{L_1} \cup \overline{L_2}$

If we assume $L_1 = \text{CFL}$ then $\overline{L_1} = \text{CFL}$

If we assume $L_2 = \text{CFL}$ then $\overline{L_2} = \text{CFL}$

then $\overline{L_1} \cup \overline{L_2}$ should be Context-free and $\overline{L_1 \cap L_2}$

should be contextfree which

means $L_1 \cap L_2$ should be CFL because but

we have seen that $L_1 \cap L_2$ is not CFL. so our assumption is wrong

CFL is not closed under

That the complementation of CFL is

Context-free

complementation

15.

$L_1 \cap L_2$

$\Rightarrow L_1/L_2$ means right quotient of L_1 with L_2

$$L_1/L_2 = \{x : xy \in L_1, \text{ for some } y \in L_2\}$$

$$L_1 = \{01, 001, 101, 0001, 1101\}$$

$$L_2 = \{01\}$$

$$L_1/L_2 = \{\epsilon, 0, 1, 00, 11\} \Rightarrow \{\epsilon, 0, 1, 00, 11\}$$

01	001	101	0001	1101
01	01	01	01	01
$\in$	0	1	00	11

Now, $L_1 = 10^*1$

$L_2 = 0^*1$

$$L_1/L_2 = \left\{ \begin{matrix} 11 & 101 & 1001 & 10001 & \dots \\ \swarrow & \searrow & \searrow & \searrow & \\ 01 & 001 & 0001 & \dots \end{matrix} \right\}$$

$$L_1/L_2 = \{1, 10, 100, 1000, \dots\}$$

$$L_1/L_2 = 10^*$$

$$\frac{11}{01} = \phi, \frac{101}{11} = \phi$$

$\Rightarrow 11$ doesn't match with 101

→ Some of the decidable problems of the context free languages are
Emptiness, finiteness, Membership

1) Membership- CYK Algorithm

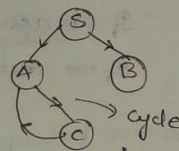
2) Emptiness — If you find starting symbol is useless then the lang is Empty.

3) Finiteness — Take Grammar 'G' and apply simplification and draw dependency Graph, If Graph has cycle then the lang is Infinite.

Say, the simplified Grammar is

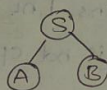
$S \rightarrow AB$
 $A \rightarrow aC|a$
 $C \rightarrow aA|b$
 $B \rightarrow a$

Dependency Graph



∴ lang is Infinite

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$



∴ No cycle ⇒ the language is finite

PROPERTIES OF REGULAR LANGUAGES

CLOSED UNDER UNION, INTERSECTION, CONCATENATION, COMPLEMENTATION

KLEEN CLOSURE

1) $L_1 \cup L_2 \Rightarrow L_1$ has Regular expression R_1

L_2 has Regular expression R_2

∴ $L_1 \cup L_2 = [R_1 + R_2] = \text{RE (Regular language)}$

2) $L_1 \cap L_2 \Rightarrow L_1 \rightarrow R_1$
 $L_2 \rightarrow R_2$
 $L_1 \cap L_2 = [R_1 \cdot R_2] = \text{Regular language}$

3. $L_1 \cdot L_2 =$

→ Some of the decidable problems of the context free languages are
Emptiness, finiteness, Membership

1) Membership- CYK Algorithm

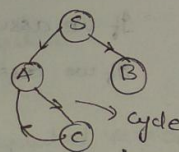
2) Emptiness — If you find starting symbol is useless then the lang is Empty.

3) Finiteness — Take Grammar 'G' and apply simplification and draw dependency Graph, If Graph has cycle then the lang is Infinite.

Say, the simplified Grammar is

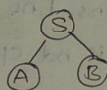
$S \rightarrow AB$
 $A \rightarrow aC|a$
 $C \rightarrow aA|b$
 $B \rightarrow a$

Dependency Graph



∴ lang is Infinite

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$



∴ No cycle ⇒ the language is finite

PROPERTIES OF REGULAR LANGUAGES

CLOSED UNDER UNION, INTERSECTION, CONCATENATION, COMPLEMENTATION

KLEEN CLOSURE

1) $L_1 \cup L_2 \Rightarrow L_1$ has Regular expression R_1

L_2 has Regular expression R_2

∴ $L_1 \cup L_2 = [R_1 + R_2] = \text{RE (Regular language)}$

2) $L_1 \cap L_2 \Rightarrow L_1 \rightarrow R_1$
 $L_2 \rightarrow R_2$
 $L_1 \cap L_2 = [R_1 \cdot R_2] = \text{Regular language}$

3. $L_1 \cdot L_2 =$

1. $L_1^* \Rightarrow R_1$ is the RE for L_1

(76)

R_1^* is the RE of L_1^* $\therefore R_1^*$ is Regular Expression

(77)

$\therefore$ Regular languages are closed under closure

5. $\bar{L}_1 = \Sigma^* - L_1 \rightarrow$ Language L_1 has DFA [Given L_1 is Regular]

$\Rightarrow$ Complement the DFA then we get DFA accepting $\bar{L}_1$

$\Rightarrow$ Since L_1 has DFA it is Regular

6. closed under difference and Reversal

$$\begin{aligned} L_1 - L_2 &\Rightarrow L_1 - L_2 \text{ can be written as } L_1 - L_2 = L_1 \cap \bar{L}_2 \\ &\quad \downarrow \quad \downarrow \\ &= RL \cap RL \\ &= \underline{RL} \end{aligned}$$



L_1 is regular language $\Rightarrow L_1^R$ should be Regular

$\therefore \Rightarrow L_1 \rightarrow$ has DFA

$\Rightarrow$ Reverse the DFA then we get DFA accepting Reverse of the language

7. closed under Homomorphism and Inverse Homomorphism

$$\Sigma = \{a, b\} \quad \Gamma = \{0, 1, 2\}$$

$$h(a) = 01$$

$$h(b) = 112$$

$$\text{Let } L = a^*b$$

$$= (01)^*112 \Rightarrow \text{Regular Expression}$$

$\therefore$ closed homomorphism

$$\Sigma = \{a, b\} \quad \Gamma = \{0, 1, 2\}$$

$$\text{Let } L = \{ababab\}$$

$$\frac{ababab}{1 \quad 1 \quad 0}$$

$$p(0) = a$$

$$p(1) = ab$$

$$p(2) = ba$$

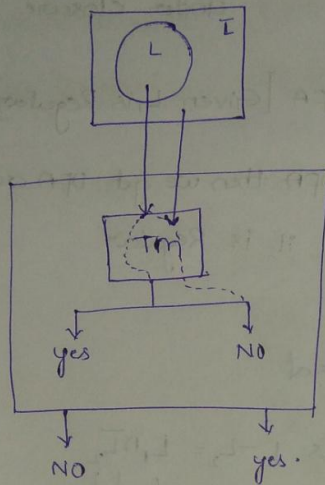
$$h^{-1}(L) = \{110022, 102\}$$

$$\frac{ababab}{1 \quad 1 \quad 0}$$

8. NOT CLOSED UNDER INFINITE UNION.

→ The complement of Recursive language is Recursive

Say there is a Recursive Language then it means there is a HTM



This TM just works in opposite way. which means if the inner TM says NO (the present in L) string is not then the outer TM says yes the string is present in $\bar{L}$ and viceversa.

∴ we got a halting TM that halts for $\bar{L}$ also. ∴ THE COMPLEMENT OF RECURSIVE LANGUAGE IS RECURSIVE.